IndustryApp – Teknik Tasarım ve Uygulama Rehberi

Sep 30, 2026 · @Bahadır İhsan Herdem

0. Kararlar ve dokümanın kullanımı

IndustryApp, demo verisi olmadan, ilk günden üretimde çalışacak şekilde tasarlanıyor. Bu doküman projenin tek kaynağıdır. Kod bu dokümana göre yazılır, PA1–PA3 dokümanları ve UML diyagramları da bu dokümandan kesilerek hazırlanır.

0.1 Veri stratejisi: seed yok, referans veri var

Sistemdeki veri iki türe ayrılıyor ve ikisine farklı davranıyoruz:

- **Referans veri:** Sistemin çalışması için var olması zorunlu, kullanıcıların üretmediği veriler. Roller, hasar kategori ağacı, araç türleri, sözleşme metinleri. Bunlar migration ile birlikte çalıştırılan sürümlü bir SQL betiğiyle (`database/reference-data.sql`) eklenir. Sebebi: üretim veritabanı boş açıldığında kayıt ekranındaki uzmanlık listesi ya da rapor sihirbazındaki hasar listesi boş kalırsa sistem kullanılamaz.
- **İşlem verisi:** Kullanıcılar, raporlar, teklifler, bildirimler. Bunlar yalnızca uygulamanın kendi ekranlarından oluşur. Müşteri ve usta kayıt ekranından kaydolur, rapor sihirbazdan açılır, teklif formdan verilir. Bildirimler ve eşleştirme gerçek olaylarla tetiklenir. Hiçbir seeder sınıfı, sahte kullanıcı veya demo endpoint'i yoktur.
- **İlk admin:** Admin hesabı herkese açık kayıtla oluşturulamaz. İlk admin normal müşteri olarak kaydolur, ardından tek satırlık SQL ile Admin rolü atanır (bkz. Bölüm 6.9). Şifre Identity'nin kendi hash'iyle oluştuğu için SQL'de şifre yazmak gerekmez.

0.2 Alınan kararlar

No	Konu	Karar	Gerekçe
K-01	Mimari stil	Modüler monolit, Clean Architecture (4 proje)	2 kişilik ekip, tek transaction içinde tutarlılık; modüller olayla haberleştiği için ileride servise bölünebilir
K-02	Modüller arası iletişim	In-process event bus (Publish-Subscribe)	Rapor modülü eşleştirmeyi bilmez; yeni dinleyici eklemek mevcut kodu değiştirmez

No	Konu	Karar	Gerekçe
K-03	Backend	.NET 8 Web API	Ekip deneyimi; mobil uygulama da aynı API'yi kullanacak (API-first)
K-04	Frontend	React + TypeScript (Vite)	Ekip tercihi
K-05	Veritabanı	PostgreSQL 16	Kısmi unique index, güçlü JSON ve coğrafi sorgu desteği
K-06	Teklif modeli	Kapalı teklif: usta yalnızca kendi teklifini görür	Fiyat kırma ve anlaşmalı fiyat riskini önler; kural backend sorgusunda uygulanır
K-07	Veri	Seed yok; referans veri SQL betiğiyle, işlem verisi ekranlardan	Proje üretime hazır teslim edilecek ve ürün olarak devam edecek
K-08	Usta onayı	Yeni usta hesabı `PendingApproval` durumunda açılır, admin onaylar	Üretimde sahte usta hesabı riskine karşı; onaysız usta eşleştirmeye girmez
K-09	Bildirim kanalları	Uygulama içi (SignalR) + e-posta (SMTP); SMS arayüzü hazır, sağlayıcı sonra	Gerçek kanallar; SMS sağlayıcısı maliyet gerektirdiği için Adapter arkasında bekler
K-10	Dosya depolama	Sunucu diskinde Docker volume, `IFileStorage` arkasında	Kurulumu en hızlısı; S3/MinIO'ya geçiş tek sınıf
K-11	Kimlik doğrulama	ASP.NET Core Identity + JWT access token + refresh token	Web ve mobil aynı mekanizmayı kullanır
K-12	İsimlendirme	Her servisin bir `I` önekli interface'i var: `ReportService` ↔ `IReportService`	Katmanlar somut sınıfa değil sözleşmeye bağlanır; test ve değişim kolaylaşır
K-13	Controller yapısı	Kaynak başına ayrı ve ince controller; controller'lar birbirini çağırmaz	Tek dev controller yerine sorumluluk ayrımı; ortak iş application servisinde

0.3 Varsayılan değerler (appsettings'ten okunur, koda gömülmez)

Ayar	Varsayılan	Açıklama
`Offers:SelectionUndoWindowMinutes`	10	Müşteri teklif seçimini bu süre içinde geri alabilir
`Reports:ExpireAfterHours`	72	Teklif seçilmeyen rapor bu süre sonunda `Expired` olur
`Matching:MaxMechanicsPerReport`	20	Bir rapor en fazla bu kadar ustaya iletilir
`Matching:DefaultRadiusKm`	15	Mesafe stratejisi seçildiğinde kullanılır
`Photos:MaxCountPerReport`	8	Rapor başına fotoğraf sınırı
`Photos:MaxFileSizeMb`	10	Tek fotoğraf yükleme sınırı
`Photos:MaxWidthPx`	1920	İşleme hattındaki yeniden boyutlandırma sınırı
`Jwt:AccessTokenMinutes`	30	Access token ömrü
`Jwt:RefreshTokenDays`	14	Refresh token ömrü

Bu değerlerden farklı istediğin olursa yalnızca bu tablo ve `appsettings.json` değişir.

1. Proje özeti

IndustryApp, kaza sonrası hasar onarımı için müşteriyle sanayi ustasını buluşturan bir ters açık artırma pazaryeridir. Müşteri hasarı fotoğraf ve adım adım kriterlerle raporlar, sistem uygun ustaları olay tabanlı olarak bulup bilgilendirir, ustalar kapalı teklif verir, müşteri birini seçer ve iletişim bilgileri yalnızca eşleşme anında açılır.

Hedef kitle: Kaza geçirmiş bireysel araç sahipleri (müşteri) ve kaporta, boya, mekanik gibi alanlarda çalışan sanayi esnafı (usta). Üçüncü aktör sistemi işleten yöneticidir (admin).

Kapsam dahil:

1. Rol bazlı kayıt ve giriş (müşteri, usta), usta hesabının admin onayı
2. Adım adım kaza raporu oluşturma: fotoğraf, hasar bölgeleri, araç türü, konum
3. Rapor oluşunca uygun ustaların otomatik bulunması ve bildirilmesi
4. Ustanın kapalı teklif vermesi, fiyat kırılımı ile
5. Müşterinin teklifleri anlık görmesi, sıralaması ve birini seçmesi

6. Seçim sonrası iletişim bilgilerinin karşılıklı açılması, süre içinde seçimi geri alma
7. Uygulama içi ve e-posta bildirimleri
8. Admin: usta onayı, kategori yönetimi

Kapsam hariç (Future Work): Ödeme ve komisyon, puanlama ve yorum, uygulama içi mesajlaşma, SMS sağlayıcısı entegrasyonu, mobil uygulama, sigorta şirketi entegrasyonu, harita üzerinde usta arama.

Başarı ölçütleri:

1. Dört use case uçtan uca, üretim ortamında gerçek kullanıcılarla çalışır.
2. Rapor oluşturma isteği, eşleştirme ve bildirim süresinden bağımsız olarak 500 ms altında yanıt verir.
3. Eşleşmemiş bir kullanıcı hiçbir API yanıtında karşı tarafın gerçek telefon numarasını göremez.
4. Bir raporda aynı anda iki kabul edilmiş teklif oluşamaz (veritabanı kısıtıyla garanti).

2. Teknoloji yığını

Katman	Teknoloji	Kullanım amacı
Backend çatısı	.NET 8, ASP.NET Core Web API	REST API, DI, middleware
ORM	Entity Framework Core 8 + Npgsql	Veri erişimi, migration
İsimlendirme	EFCore.NamingConventions	C# PascalCase → PostgreSQL snake_case otomatik
Kimlik	ASP.NET Core Identity + JwtBearer	Kullanıcı, rol, şifre hash, token
Gerçek zamanlı	SignalR	Usta bildirimi, yeni teklif, seçim bildirimleri
Doğrulama	FluentValidation	Request modellerinin kuralları
Dayanıklılık	Polly (Microsoft.Extensions.Http.Resilience / Polly v8)	Circuit Breaker + Retry, e-posta ve SMS kanallarında
Görsel işleme	SixLabors.ImageSharp	Yeniden boyutlandırma, sıkıştırma, EXIF temizleme

Katman	Teknoloji	Kullanım amacı
E-posta	MailKit	SMTP üzerinden gönderim
Loglama	Serilog (Console + File sink)	Yapısal log, istek korelasyonu
API dokümantasyonu	Swashbuckle (Swagger)	Endpoint keşfi, test
Test	xUnit, NSubstitute, FluentAssertions, Testcontainers.PostgreSql	Birim ve entegrasyon testi
Veritabanı	PostgreSQL 16	Ana veri deposu
Frontend	React 18, TypeScript, Vite	SPA
Frontend yönlendirme	React Router 6	Rol bazlı route'lar
Sunucu verisi	TanStack Query 5 + Axios	Önbellek, yeniden deneme, token yenileme
Formlar	React Hook Form + Zod	Sihirbaz formları ve doğrulama
Arayüz bileşenleri	MUI 5	Hazır form, tablo, dialog, bildirim bileşenleri
Anlık istemci	@microsoft/signalr	Hub bağlantısı
Dağıtım	Docker, Docker Compose, Caddy	Konteyner, ters vekil, otomatik HTTPS

Not: MediatR kullanılmıyor. Lisans modeli değiştiği için olay dağıtımı `IEventDispatcher` ile kendimiz yazılıyor. Bu, Observer/Mediator desenini sunumda kendi kodumuz üzerinden göstermemizi de sağlıyor.

3. Repo ve solution yapısı

Backend, frontend, veritabanı betikleri, dağıtım dosyaları ve dokümanlar tek bir repoda durur. Backend 4 kaynak projeden ve 2 test projesinden oluşur. Her modül (Accounts, Catalog, Reports, Matching, Offers, Contacts, Notifications, Admin) her katmanda aynı isimli klasöre sahiptir, böylece bir özelliğe ait kodun yeri tahmin edilebilir.

```
IndustryApp/
├── database/
|   ├── reference-data.sql          → roller, kategoriler, araç türleri,
sözleşmeler
|   ├── admin-bootstrap.sql         → ilk admin'e rol atama
|   └── dev/uc04-dev-data.sql       → yalnızca yerel, F4 boyunca; F6 sonunda
silinir
├── deploy/
|   ├── Caddyfile                   → HTTPS + ters vekil
|   └── .env.example                → üretim ortam değişkenleri şablonu
├── docs/
|   ├── IndustryApp_Teknik_Tasarim.md
|   ├── PA1/  PA2/  PA3/
|   └── diagrams/
├── src/
|   ├── backend/
|   |   ├── IndustryApp.sln
|   |   ├── IndustryApp.Domain/
|   |   |   ├── Common/             → BaseEntity, AuditableEntity,
IDomainEvent, IHasDomainEvents, DomainException
|   |   |   ├── Accounts/           → CustomerProfile, MechanicProfile,
MechanicSpecialty, MechanicApprovalStatus,
|   |   |   |                          LegalDocument, UserConsent, RoleNames
|   |   |   ├── Catalog/            → IDamageComponent, DamageCategory,
VehicleType
|   |   |   ├── Reports/            → AccidentReport, AccidentReportBuilder,
ReportDamageItem, ReportPhoto,
|   |   |   |   |                      ReportMatch, ReportStatus,
ReportStatusHistory, GeoLocation
|   |   |   |   └── States/         → IReportState, OpenState,
OfferSelectedState, CompletedState,
|   |   |   |                          CancelledState, ExpiredState,
ReportStateFactory
|   |   |   ├── Offers/             → Offer, OfferStatus, OfferPriceItem,
PriceItemType
|   |   |   |   └── Pricing/        → IPriceComponent, BasePrice,
PriceComponentDecorator,
|   |   |   |                          PartsCost, PaintCost, WarrantyCost
|   |   |   ├── Contacts/           → IContactInfo, ContactInfo,
ContactInfoProxy
|   |   |   ├── Notifications/      → Notification, NotificationType,
NotificationChannelType
|   |   |   └── Events/             → AccidentReportCreatedEvent,
MechanicsMatchedEvent, OfferSubmittedEvent,
|   |   |                              OfferSelectedEvent,
OfferSelectionUndoneEvent, ReportExpiredEvent,
```

```
|   |   |                              ReportCompletedEvent,
ReportCancelledEvent, OfferUpdatedEvent, OfferWithdrawnEvent,
MechanicApprovedEvent, MechanicRejectedEvent
|   |   ├── IndustryApp.Application/
|   |   |   ├── Abstractions/       → I*Repository, IUnitOfWork,
IEventDispatcher, IEventHandler<T>,
|   |   |   |                          ICurrentUser, IDateTimeProvider,
IFileStorage, IImageProcessor,
|   |   |   |                          INotificationChannel,
IRealtimePublisher, ITokenService, IIdentityService
|   |   |   ├── Common/             → Result, Error, PagedResult, exception
türleri, Options sınıfları
|   |   |   ├── Accounts/           → IAuthService, AuthService,
IAccountService, AccountService,
|   |   |   |   ├── Factories/      → IUserProfileFactory (+
Customer/Mechanic), IDashboardFactory (+ 3 aile)
|   |   |   |   ├── Contracts/      → request/response modelleri
|   |   |   |   └── Validators/
|   |   |   ├── Catalog/            → ICatalogService, CatalogService,
Contracts
|   |   |   ├── Reports/            → IReportService, ReportService,
Contracts, Validators
|   |   |   |   └── Photos/         → IPhotoService, PhotoService
|   |   |   ├── Matching/           → IMatchingService, MatchingService
|   |   |   |   ├── Strategies/     → IMatchingStrategy,
CityMatchingStrategy, RadiusMatchingStrategy,
|   |   |   |   |                      IMatchingStrategyResolver
|   |   |   |   ├── Filters/        → IMechanicFilter, MechanicFilterBase,
ApprovedAndActiveFilter,
|   |   |   |   |                      SpecialtyFilter, AvailabilityFilter,
DailyCapacityFilter
|   |   |   |   └── Handlers/       → AccidentReportCreatedMatchingHandler
|   |   |   ├── Offers/             → IOfferService, OfferService,
IOfferSelectionService, OfferSelectionService
|   |   |   |   ├── Commands/       → IReportCommand, SelectOfferCommand,
IReportCommandFactory
|   |   |   |   ├── Sorting/        → IOfferSortStrategy, PriceAscending,
NewestFirst, FastestCompletion
|   |   |   |   ├── Contracts/
|   |   |   |   └── Validators/
|   |   |   ├── Contacts/           → IContactService, ContactService,
Contracts
|   |   |   ├── Notifications/      → INotificationService,
NotificationService, Contracts
|   |   |   |   └── Handlers/       → her olay için bir bildirim handler'ı
|   |   |   ├── Admin/              → IMechanicApprovalService,
```

```
ICategoryAdminService (+ implementasyonlar)
|   |   |   └── DependencyInjection.cs
|   |   ├── IndustryApp.Infrastructure/
|   |   |   ├── Persistence/        → AppDbContext, Configurations/,
Repositories/, Migrations/, UnitOfWork
|   |   |   ├── Identity/           → ApplicationUser, ApplicationRole,
IdentityService, JwtTokenService,
|   |   |   |                          RefreshToken, CurrentUser
|   |   |   ├── Events/             → InProcessEventDispatcher, EventQueue,
EventProcessingBackgroundService
|   |   |   ├── Storage/            → LocalFileStorage
|   |   |   ├── Imaging/            → ImageSharpProcessor, ResizeProcessor,
CompressionProcessor, ExifStripProcessor
|   |   |   ├── Notifications/      → SmtpEmailChannel, LoggingSmsChannel,
ResilientNotificationChannel
|   |   |   ├── BackgroundJobs/     → ReportExpirationJob
|   |   |   ├── Time/               → SystemDateTimeProvider
|   |   |   └── DependencyInjection.cs
|   |   ├── IndustryApp.WebApi/
|   |   |   ├── Controllers/        → 12 controller (Bölüm 11)
|   |   |   |   └── Admin/          → AdminMechanicsController,
AdminCatalogController
|   |   |   ├── Realtime/           → NotificationHub,
SignalRRealtimePublisher
|   |   |   ├── Middleware/         → GlobalExceptionHandler,
CorrelationIdMiddleware
|   |   |   ├── Extensions/         → Swagger, Auth, CORS, RateLimiting
kayıtları
|   |   |   ├── Program.cs
|   |   |   └── appsettings.json / appsettings.Production.json
|   |   └── tests/
|   |       ├── IndustryApp.UnitTests/
|   |       └── IndustryApp.IntegrationTests/
|   └── frontend/
|       └── industryapp-web/        → Bölüm 14
├── docker-compose.yml
├── .editorconfig
├── .gitignore
└── README.md
```

3.1 Projeler arası referanslar

Proje	Referans verdiği projeler	Neden
Domain	Hiçbiri	Saf iş modeli; EF, ASP.NET, Identity bilmez

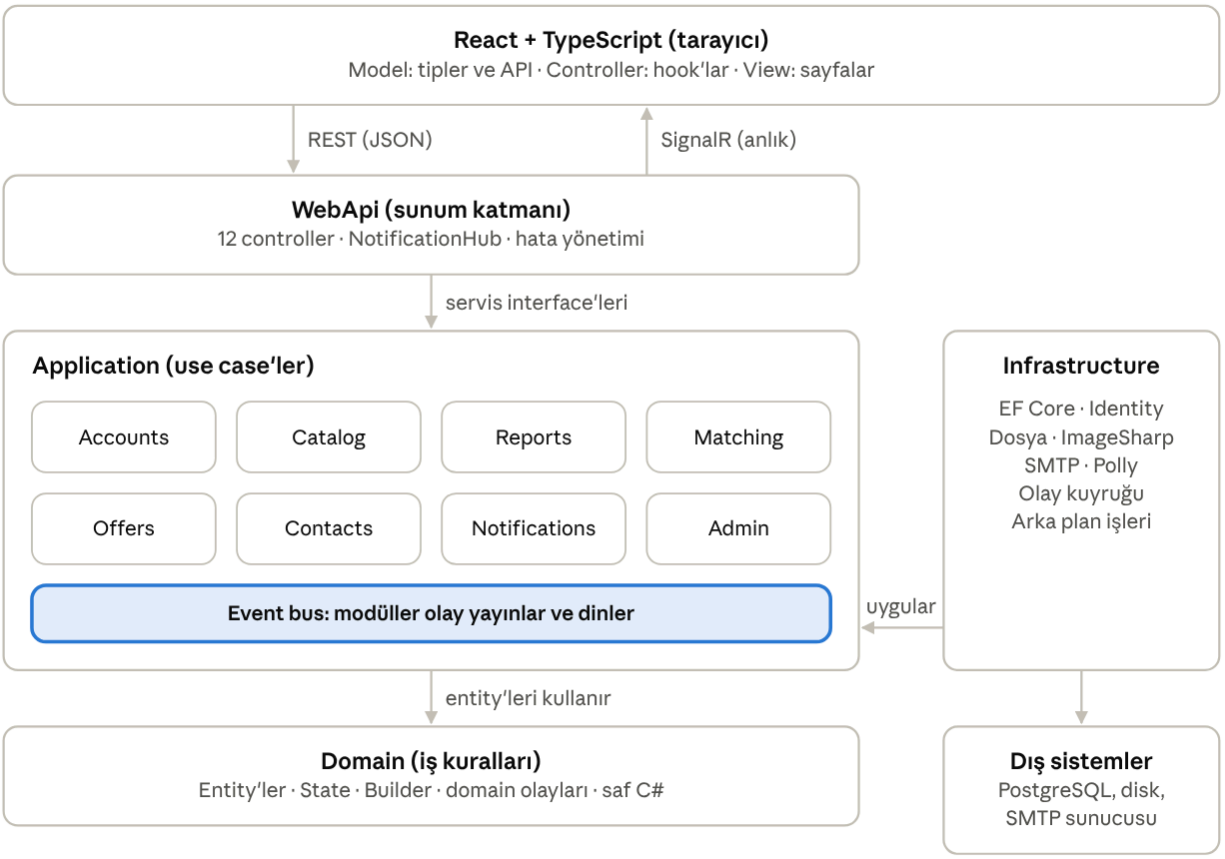
Proje	Referans verdiği projeler	Neden
Application	Domain	Use case'ler domain nesnelerini kullanır, dış dünyayı yalnızca interface olarak tanır
Infrastructure	Application	Application'daki interface'leri implemente eder (Domain dolaylı gelir)
WebApi	Application, Infrastructure	Composition root: DI kayıtları burada yapılır; controller'lar yalnızca Application interface'lerini kullanır
UnitTests	Domain, Application	Veritabanısız, sahte interface'lerle test
IntegrationTests	WebApi	Gerçek PostgreSQL konteyneri üzerinde uçtan uca test

WebApi'nin Infrastructure'a referansı yalnızca `Program.cs` içindeki `AddInfrastructure()` çağrısı için vardır. Bir controller'ın Infrastructure'daki bir sınıfı doğrudan kullanması kural ihlalidir ve kod incelemesinde reddedilir.

4. Mimari ve katmanlar arası bağlantı

Birincil mimari örüntü Layered (Clean Architecture); modüller arası iletişimde Publish-Subscribe, sunumda dağıtık MVC kullanılır. Bağımlılıklar her zaman içe doğrudur: WebApi → Application → Domain. Infrastructure, Application'ın tanımladığı interface'leri dışarıdan doldurur.

İstekler içe doğru akar; modüller olayla haberleşir



IndustryApp katmanları · 4 katman, 8 modül, 1 olay veriyolu

WebApi yalnızca Application'daki servis interface'lerini çağırır; Infrastructure bu interface'leri dışarıdan uygular ve Domain hiçbir katmanı tanımaz.

4.1 Katmanların sorumlulukları

Katman	Ne yapar	Ne yapmaz
Domain	Entity'ler, iş kuralları, durum makinesi, domain olayları, değer nesneleri	Veritabanına yazmaz, HTTP bilmez, tarih için `DateTime.UtcNow` çağırmaz (zaman parametre olarak gelir)
Application	Use case'leri yönetir: yetki kontrolü, doğrulama, repository'den okuma, domain metodunu çağırma, kaydetme, olay yayını	SQL yazmaz, dosya sistemine erişmez, SignalR veya SMTP bilmez

Katman	Ne yapar	Ne yapmaz
Infrastructure	EF Core, Identity, dosya depolama, e-posta, görsel işleme, olay kuyruğu, arka plan işleri	İş kuralı içermez; "teklif seçilebilir mi" kararını vermez
WebApi	HTTP isteğini alır, request modeline bağlar, ilgili servis interface'ini çağırır, sonucu HTTP koduna çevirir	İş mantığı içermez; başka controller çağırmaz; DbContext kullanmaz

4.2 Bir isteğin yolculuğu (örnek: teklif seçimi)

1. React'teki `OfferList` view'ı "Bu teklifi seç" butonunda `useSelectOffer` hook'unu (controller) çağırır.
2. Hook, `offerSelectionApi.select(reportId, offerId)` ile `POST /api/reports/{reportId}/selection` isteği atar.
3. `OfferSelectionController.Select` yalnızca `IOfferSelectionService.SelectAsync(ISelectOfferRequest)` çağırır.
4. `OfferSelectionService` şunları yapar: `ICurrentUser` ile müşterinin raporun sahibi olduğunu doğrular, `IAccidentReportRepository` ile raporu ve teklifleri yükler, `IReportCommandFactory` ile `SelectOfferCommand` üretip `Execute()` eder.
5. Komut, raporun o anki `IReportState` nesnesine "teklif seç" der. `OpenState` bunu kabul edip raporu `OfferSelected` yapar; başka bir durumdaysa `DomainException` fırlatılır.
6. Durum geçişi `ReportStatusHistory` kaydı ve `OfferSelectedEvent` domain olayı üretir.
7. `IUnitOfWork.SaveChangesAsync()` tek transaction içinde kaydeder. Kısmi unique index ikinci bir kabulü veritabanı seviyesinde reddeder.
8. Kayıt başarılı olunca `IEventDispatcher`, biriken olayları kuyruğa yazar ve HTTP yanıtı hemen döner.
9. Arka planda `OfferSelectedEvent` handler'ları çalışır: kazanan ustaya ve diğer ustalara bildirim, SignalR ile müşterinin ekranına güncelleme.

4.3 Olay akışı

Olaylar transaction commit edildikten sonra yayınlanır. Böylece bildirimi alan kişi kaydedilmemiş bir kaydı açmaya çalışmaz. Kuyruğa alma ve arka planda işleme, HTTP yanıtının eşleştirme veya e-posta gönderimini beklemesini engeller.

Olay	Yayınlayan	Dinleyen handler'lar
`AccidentReportCreatedEvent`	ReportService	Matching: `AccidentReportCreatedMatchingHandler`
`MechanicsMatchedEvent`	MatchingService	Notifications: eşleşen ustalara "yeni iş" bildirimi
`OfferSubmittedEvent`	OfferService	Notifications: müşteriye "yeni teklif"; Realtime: rapor detay ekranına canlı ekleme
`OfferSelectedEvent`	OfferSelectionService	Notifications: kazanan ustaya "seçildiniz", diğerlerine "başka teklif seçildi"
`OfferSelectionUndoneEvent`	OfferSelectionService	Notifications: ilgili ustalara "rapor yeniden teklife açıldı"
`ReportCompletedEvent`	ReportService	Notifications: ustaya "iş tamamlandı olarak işaretlendi"
`ReportExpiredEvent`	ReportExpirationJob	Notifications: müşteriye "raporunuzun süresi doldu"
`MechanicApprovedEvent`	MechanicApprovalService	Notifications: ustaya e-posta ile "hesabınız onaylandı"

4.4 İsimlendirme ve sözleşme kuralları

Her servis bir interface arkasındadır ve interface adı sınıf adının başına `I` eklenerek oluşturulur. Katmanlar birbirine her zaman interface üzerinden bağlanır.

Öğe	Kural	Örnek
Servis	`<Konu>Service` + `I<Konu>Service`	`ReportService` ↔ `IReportService`
Repository	`<Entity>Repository` + `I<Entity>Repository`	`OfferRepository` ↔ `IOfferRepository`
Metot parametresi (girdi)	`I<Eylem><Konu>Request` interface'i + onu uygulayan `record`	`ICreateReportRequest` ↔ `CreateReportRequest`
Metot dönüşü (çıktı)	`<Konu><Amaç>Response` record	`ReportDetailResponse`, `OfferListItemResponse`
Strateji / filtre / komut	Desen rolünü ad sonunda taşır	`CityMatchingStrategy`, `SpecialtyFilter`, `SelectOfferCommand`

Öğe	Kural	Örnek
Olay	Geçmiş zaman + `Event`	`OfferSelectedEvent`
Olay handler'ı	`<Olay>` + `<İş>Handler`	`OfferSelectedNotificationHandler`
Controller	Kaynak adının çoğulu + `Controller`	`ReportsController`, `OffersController`
Asenkron metot	`Async` soneki, son parametre `CancellationToken ct`	`SelectAsync(ISelectOfferRequest request, CancellationToken ct)`

Neden request'ler de interface: Aynı servis metodu hem web hem ileride mobil hem de arka plan işi tarafından çağrılabilir. Servis somut `record`'a değil `ISelectOfferRequest` sözleşmesine bağlı olursa, her çağıran kendi modelini bu sözleşmeye uydurarak gönderebilir. Validator'lar da interface üzerine yazıldığı için tek bir kural seti tüm çağıranlara uygulanır.

Controller'lar arası çağrı kuralı: Controller'lar birbirini asla çağırmaz. İki controller aynı işe ihtiyaç duyuyorsa o iş bir application servisine taşınır ve iki controller da aynı servis interface'ini kullanır. Örneğin `ReportsController` (müşteri) ve `MechanicJobsController` (usta) rapor detayını farklı yetkilerle gösterir ama ikisi de `IReportService`'i çağırır. Bir modülün başka modüle ihtiyacı varsa bu ya servis interface'i enjekte edilerek ya da olay yayınlanarak karşılanır; ikincisi tercih edilir.

5. Domain modeli

Domain 15 entity ve 2 durum makinesinden oluşur. Kullanıcı hesabı (e-posta, şifre, telefon) Identity'ye aittir ve Infrastructure'daki `ApplicationUser` sınıfında durur. Domain kullanıcıyı yalnızca `Guid UserId` olarak tanır; böylece domain Identity kütüphanesine bağımlı olmaz.

5.1 Entity'ler ve ilişkileri

Entity	Aggregate	Temel alanlar	İlişkiler
`ApplicationUser` (Infrastructure)	Kök	Id, Email, PhoneNumber, FullName, IsActive	1–1 CustomerProfile veya MechanicProfile; 1–N UserConsent, RefreshToken, Notification
`CustomerProfile`	Kök	UserId, City, District	1–N AccidentReport

Entity	Aggregate	Temel alanlar	İlişkiler
`MechanicProfile`	Kök	UserId, WorkshopName, TaxNumber, City, District, Address, Location, ApprovalStatus, IsAvailable, DailyCapacity	1–N MechanicSpecialty, Offer, ReportMatch
`MechanicSpecialty`	MechanicProfile	MechanicId, DamageCategoryId	N–1 DamageCategory
`DamageCategory`	Kök (ağaç)	Id, ParentId, Code, Name, Level, SortOrder, IsActive	Kendi kendine 1–N (Composite)
`VehicleType`	Kök	Id, Code, Name	1–N AccidentReport
`LegalDocument`	Kök	Code, Version, TargetRole, Title, Body	1–N UserConsent
`UserConsent`	ApplicationUser	UserId, LegalDocumentId, AcceptedAt, IpAddress	N–1 LegalDocument
`AccidentReport`	Kök	Id, CustomerId, VehicleTypeId, Description, Location, City, District, Status, SelectedOfferId, OfferSelectedAt, ExpiresAt	1–N ReportDamageItem, ReportPhoto, ReportMatch, Offer, ReportStatusHistory
`ReportDamageItem`	AccidentReport	ReportId, DamageCategoryId, Severity, Note	N–1 DamageCategory
`ReportPhoto`	AccidentReport	ReportId, StorageKey, ContentType, SizeBytes, Width, Height, SortOrder	—

Entity	Aggregate	Temel alanlar	İlişkiler
`ReportMatch`	AccidentReport	ReportId, MechanicId, StrategyCode, DistanceKm, NotifiedAt, ViewedAt	N–1 MechanicProfile
`ReportStatusHistory`	AccidentReport	ReportId, FromStatus, ToStatus, CommandName, ActorUserId, Snapshot (jsonb), OccurredAt	—
`Offer`	Kök	Id, ReportId, MechanicId, TotalPrice, Currency, EstimatedDays, Note, Status	1–N OfferPriceItem
`OfferPriceItem`	Offer	OfferId, ItemType, Description, Amount, SortOrder	—
`Notification`	Kök	UserId, Type, Title, Body, PayloadJson, IsRead, CreatedAt, ReadAt	—

Aggregate sınırı neden böyle: `Offer` ayrı bir kök çünkü ustalar aynı rapora aynı anda teklif verir; teklifleri raporun içine koysaydık her teklif tüm raporu kilitlerdi. Buna karşılık seçim işlemi raporun durumunu değiştirdiği için `AccidentReport.SelectOffer` üzerinden yapılır ve teklif durumları aynı transaction içinde güncellenir.

5.2 Kaza raporu durum makinesi (State deseni)

Rapor durumu veritabanında `status` kolonunda metin olarak saklanır. Rapor yüklendiğinde `ReportStateFactory`, bu değere karşılık gelen `IReportState` nesnesini üretir. Rapor üzerindeki her işlem o anki state nesnesine devredilir, böylece `if (status == ...)` kontrolleri kod geneline dağılmaz.

Durum	Kabul ettiği işlemler	Geçiş
`Open`	Teklif al, teklif seç, iptal et, süresi dol	→ OfferSelected, Cancelled, Expired
`OfferSelected`	Seçimi geri al (süre içinde), tamamlandı işaretle, iptal et	→ Open, Completed, Cancelled
`Completed`	Hiçbiri (son durum)	—
`Cancelled`	Hiçbiri (son durum)	—
`Expired`	Hiçbiri (son durum)	—

Kabul edilmeyen işlem çağrıldığında state nesnesi `DomainException("REPORT_INVALID_STATE")` fırlatır. Örneğin `OfferSelected` durumundaki bir rapora usta teklif göndermeye çalışırsa API `409 Conflict` döner.

5.3 Teklif durumları

Durum	Anlamı	Nereden gelir
`Pending`	Müşterinin kararını bekliyor	Teklif oluşturulunca
`Accepted`	Müşteri bu teklifi seçti	Seçim komutu
`NotSelected`	Başka teklif seçildi	Seçim komutu; geri alınırsa tekrar `Pending`
`Withdrawn`	Usta teklifini geri çekti	Yalnızca `Pending` iken
`Closed`	Rapor iptal edildi veya süresi doldu	Rapor son duruma geçince

5.4 Temel domain metotları

Sınıf	Metot	Kural
`AccidentReportBuilder`	`WithCustomer`, `WithVehicleType`, `AddDamage`, `AddPhoto`, `AtLocation`, `WithDescription`, `Build(now, ttl)`	En az 1 hasar, 1 fotoğraf, konum ve araç türü yoksa `Build` hata verir

Sınıf	Metot	Kural
`AccidentReport`	`CanReceiveOfferFrom(mechanicId)`	Rapor `Open` ve ustayla eşleşmiş olmalı
`AccidentReport`	`SelectOffer(offer, actorId, now)`	Teklif bu rapora ait ve `Pending` olmalı
`AccidentReport`	`UndoSelection(actorId, now, window)`	`now - OfferSelectedAt <= window` olmalı
`AccidentReport`	`MarkCompleted`, `Cancel`, `Expire`	State nesnesi karar verir
`AccidentReport`	`IsMatchedWith(userId)`	Sahibi müşteri veya kabul edilmiş teklifin ustası mı
`Offer`	`Create(report, mechanicId, priceComponent, ...)`	Toplam fiyat Decorator zincirinden hesaplanır, 0'dan büyük olmalı
`Offer`	`Accept`, `MarkNotSelected`, `Reopen`, `Withdraw`, `Close`	Geçersiz geçişte `DomainException`
`MechanicProfile`	`Approve`, `Reject(reason)`, `SetAvailability`	Onaysız usta eşleştirmeye girmez

6. Veritabanı

Veritabanı 21 uygulama tablosu ve Identity'nin 4 yardımcı tablosundan oluşur. Şema EF Core migration'larıyla üretilir; aşağıdaki DDL, EF configuration sınıflarının hedeflediği şemadır. Migration üretildikten sonra `dotnet ef migrations script` çıktısı bu DDL ile karşılaştırılır.

6.1 Genel kurallar

- Tablo ve kolon adları snake_case, tablo adları çoğul. `EFCore.NamingConventions` ile otomatik; Identity tabloları `OnModelCreating` içinde `users`, `roles`, `user_roles` gibi sade adlara çevrilir.
- Kullanıcıya veya API'ye görünen kayıtların anahtarı `uuid` (tahmin edilemez, mobil istemcide güvenli). Lookup tablolarının anahtarı `int` / `smallint` identity.

- Tüm zamanlar `timestamptz`, UTC yazılır. Frontend gösterimde yerel saate çevirir.
- Para `numeric(12,2)`, para birimi `char(3)`.
- Enum değerleri `varchar` olarak metin halinde saklanır ve `CHECK` kısıtıyla korunur. Sayısal enum saklamak, enum sırası değişince veriyi bozar.
- Eşzamanlılık kontrolü için PostgreSQL'in `xmin` sistem kolonu kullanılır (`UseXminAsConcurrencyToken`). İki kullanıcı aynı raporu aynı anda değiştirirse ikincisi `409 Conflict` alır.
- Silme yerine durum değişikliği kullanılır. Rapor silinmez, `Cancelled` olur; kullanıcı silinmez, `is_active = false` olur.

6.2 Tablolar ve ilişkiler

Tablo	Anahtar	Bağlandığı tablolar	İlişki
`users`	id uuid	—	Identity kullanıcı kökü
`roles`	id uuid	—	Customer, Mechanic, Admin
`user_roles`	(user_id, role_id)	users, roles	N–N
`refresh_tokens`	id uuid	users	users 1–N
`customer_profiles`	user_id	users	users 1–1
`mechanic_profiles`	user_id	users	users 1–1
`damage_categories`	id int	damage_categories	Kendi kendine 1–N (ağaç)
`mechanic_specialties`	(mechanic_id, damage_category_id)	mechanic_profiles, damage_categories	N–N
`vehicle_types`	id smallint	—	Lookup
`legal_documents`	id int	—	Lookup, sürümlü
`user_consents`	id bigint	users, legal_documents	users 1–N

Tablo	Anahtar	Bağlandığı tablolar	İlişki
`accident_reports`	id uuid	customer_profiles, vehicle_types, offers	customer 1–N; seçilen teklif 0..1
`report_damage_items`	id bigint	accident_reports, damage_categories	report 1–N
`report_photos`	id uuid	accident_reports	report 1–N
`report_matches`	(report_id, mechanic_id)	accident_reports, mechanic_profiles	N–N
`report_status_history`	id bigint	accident_reports, users	report 1–N
`offers`	id uuid	accident_reports, mechanic_profiles	report 1–N; mechanic 1–N
`offer_price_items`	id bigint	offers	offer 1–N
`notifications`	id uuid	users	users 1–N
`notification_deliveries`	id bigint	notifications	notification 1–N (kanal başına)

Identity ayrıca `user_claims`, `user_logins`, `user_tokens`, `role_claims` tablolarını oluşturur. Bunlara dokunulmaz.

6.3 Kimlik ve profil tabloları

```
CREATE TABLE users (
    id                      uuid PRIMARY KEY,
    user_name               varchar(256),
    normalized_user_name    varchar(256) UNIQUE,
    email                   varchar(256) NOT NULL,
    normalized_email        varchar(256) NOT NULL UNIQUE,
    email_confirmed         boolean      NOT NULL DEFAULT false,
    password_hash           text,
    security_stamp          text,
    concurrency_stamp       text,
    phone_number            varchar(20)  NOT NULL,
    phone_number_confirmed  boolean      NOT NULL DEFAULT false,
    two_factor_enabled      boolean      NOT NULL DEFAULT false,
    lockout_end             timestamptz,
```

```
    lockout_enabled         boolean      NOT NULL DEFAULT true,
    access_failed_count     integer      NOT NULL DEFAULT 0,
    -- IndustryApp eklemeleri
    full_name               varchar(150) NOT NULL,
    is_active               boolean      NOT NULL DEFAULT true,
    created_at              timestamptz  NOT NULL DEFAULT now(),
    last_login_at           timestamptz
);

CREATE TABLE roles (
    id                uuid PRIMARY KEY,
    name              varchar(256) NOT NULL,
    normalized_name   varchar(256) NOT NULL UNIQUE,
    concurrency_stamp  text
);

CREATE TABLE user_roles (
    user_id  uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role_id  uuid NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, role_id)
);

CREATE TABLE refresh_tokens (
    id                      uuid PRIMARY KEY,
    user_id                 uuid         NOT NULL REFERENCES users(id) ON
DELETE CASCADE,
    token_hash              varchar(128) NOT NULL UNIQUE,   -- token'ın
kendisi değil SHA-256 özeti
    expires_at              timestamptz  NOT NULL,
    created_at              timestamptz  NOT NULL DEFAULT now(),
    created_by_ip           varchar(45),
    revoked_at              timestamptz,
    replaced_by_token_hash  varchar(128)
);

CREATE TABLE customer_profiles (
    user_id     uuid PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    city        varchar(50)  NOT NULL,
    district    varchar(80),
    created_at  timestamptz  NOT NULL DEFAULT now(),
    updated_at  timestamptz
);

CREATE TABLE mechanic_profiles (
    user_id           uuid PRIMARY KEY REFERENCES users(id) ON DELETE
CASCADE,
```

```
    workshop_name     varchar(150)     NOT NULL,
    tax_number        varchar(11)      NOT NULL UNIQUE,
    city              varchar(50)      NOT NULL,
    district          varchar(80)      NOT NULL,
    address           varchar(300)     NOT NULL,
    latitude          double precision NOT NULL CHECK (latitude BETWEEN -90
AND 90),
    longitude         double precision NOT NULL CHECK (longitude BETWEEN -180
AND 180),
    approval_status   varchar(20)      NOT NULL DEFAULT 'PendingApproval'
                      CHECK (approval_status IN
('PendingApproval','Approved','Rejected','Suspended')),
    rejection_reason  varchar(500),
    approved_at       timestamptz,
    approved_by       uuid REFERENCES users(id),
    is_available      boolean          NOT NULL DEFAULT true,
    daily_capacity    smallint         NOT NULL DEFAULT 5 CHECK
(daily_capacity > 0),
    created_at        timestamptz      NOT NULL DEFAULT now(),
    updated_at        timestamptz
);
```

Neden telefon `users` tablosunda: Telefon numarası her iki rol için ortak ve Identity'nin standart alanı. İletişim bilgisi açılırken tek kaynaktan okunur, Proxy de tek yerden kontrol eder.

6.4 Katalog ve sözleşme tabloları

```
CREATE TABLE damage_categories (
    id          integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    parent_id   integer REFERENCES damage_categories(id),
    root_id     integer REFERENCES damage_categories(id),   -- 1. seviyede
kendisi; eşleştirmede hızlı uzmanlık kontrolü
    code        varchar(50)  NOT NULL UNIQUE,
    name        varchar(100) NOT NULL,
    level       smallint     NOT NULL CHECK (level BETWEEN 1 AND 3),
    sort_order  smallint     NOT NULL DEFAULT 0,
    is_active   boolean      NOT NULL DEFAULT true
);

CREATE TABLE mechanic_specialties (
    mechanic_id         uuid    NOT NULL REFERENCES
mechanic_profiles(user_id) ON DELETE CASCADE,
    damage_category_id  integer NOT NULL REFERENCES damage_categories(id),
-- yalnızca level = 1
    PRIMARY KEY (mechanic_id, damage_category_id)
```

```
);

CREATE TABLE vehicle_types (
    id          smallint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code        varchar(30) NOT NULL UNIQUE,
    name        varchar(60) NOT NULL,
    sort_order  smallint    NOT NULL DEFAULT 0,
    is_active   boolean     NOT NULL DEFAULT true
);

CREATE TABLE legal_documents (
    id            integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code          varchar(50)  NOT NULL,
    version       varchar(20)  NOT NULL,
    target_role   varchar(20)  NOT NULL CHECK (target_role IN
('Customer','Mechanic','All')),
    title         varchar(200) NOT NULL,
    body          text         NOT NULL,
    published_at  timestamptz  NOT NULL DEFAULT now(),
    is_current    boolean      NOT NULL DEFAULT true,
    UNIQUE (code, version)
);
CREATE UNIQUE INDEX ux_legal_documents_current ON legal_documents(code) WHERE
is_current;

CREATE TABLE user_consents (
    id                bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id           uuid        NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
    legal_document_id  integer     NOT NULL REFERENCES legal_documents(id),
    accepted_at        timestamptz NOT NULL DEFAULT now(),
    ip_address         varchar(45),
    UNIQUE (user_id, legal_document_id)
);
```

Uzmanlık neden yalnızca 1. seviyede: Usta "Kaporta" ya da "Boya" işi yapar; "sol ön çamurluk" diye uzmanlık seçmez. Rapordaki hasar 3. seviyede olsa bile `root_id` sayesinde tek karşılaştırmayla ilgili ustalar bulunur, ağaçta yukarı doğru özyinelemeli sorgu gerekmez.

6.5 Rapor tabloları

```
CREATE TABLE accident_reports (
    id                uuid PRIMARY KEY,
    customer_id       uuid             NOT NULL REFERENCES
customer_profiles(user_id),
```

```
    vehicle_type_id    smallint         NOT NULL REFERENCES
vehicle_types(id),
    description        varchar(1000),
    city               varchar(50)      NOT NULL,
    district           varchar(80),
    address_text       varchar(300),
    latitude           double precision NOT NULL,
    longitude          double precision NOT NULL,
    status             varchar(20)      NOT NULL DEFAULT 'Open'
                       CHECK (status IN
('Open','OfferSelected','Completed','Cancelled','Expired')),
    selected_offer_id  uuid,            -- FK offers tablosundan sonra
eklenir
    offer_selected_at  timestamptz,
    expires_at         timestamptz      NOT NULL,
    completed_at       timestamptz,
    cancelled_at       timestamptz,
    created_at         timestamptz      NOT NULL DEFAULT now(),
    updated_at         timestamptz,
    CHECK (status <> 'OfferSelected' OR selected_offer_id IS NOT NULL)
);

CREATE TABLE report_damage_items (
    id                 bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    report_id          uuid        NOT NULL REFERENCES accident_reports(id)
ON DELETE CASCADE,
    damage_category_id  integer     NOT NULL REFERENCES
damage_categories(id),
    severity           varchar(10) NOT NULL CHECK (severity IN
('Light','Medium','Heavy')),
    note               varchar(300),
    UNIQUE (report_id, damage_category_id)
);

CREATE TABLE report_photos (
    id                 uuid PRIMARY KEY,
    report_id          uuid         NOT NULL REFERENCES accident_reports(id)
ON DELETE CASCADE,
    storage_key        varchar(300) NOT NULL,   -- örn.
reports/2026/10/<reportId>/<photoId>.jpg
    original_file_name  varchar(255),
    content_type       varchar(50)  NOT NULL,
    size_bytes         integer      NOT NULL,
    width              integer      NOT NULL,
    height             integer      NOT NULL,
    sort_order         smallint     NOT NULL DEFAULT 0,
```

```
    created_at          timestamptz  NOT NULL DEFAULT now()
);

CREATE TABLE report_matches (
    report_id      uuid         NOT NULL REFERENCES accident_reports(id) ON
DELETE CASCADE,
    mechanic_id    uuid         NOT NULL REFERENCES
mechanic_profiles(user_id),
    strategy_code  varchar(30)  NOT NULL,           -- CITY, RADIUS
    distance_km    numeric(6,2),
    notified_at    timestamptz  NOT NULL DEFAULT now(),
    viewed_at      timestamptz,
    PRIMARY KEY (report_id, mechanic_id)
);

CREATE TABLE report_status_history (
    id             bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    report_id      uuid        NOT NULL REFERENCES accident_reports(id) ON
DELETE CASCADE,
    from_status    varchar(20),
    to_status      varchar(20) NOT NULL,
    command_name   varchar(60) NOT NULL,            -- SelectOfferCommand,
ExpireReport ...
    actor_user_id  uuid REFERENCES users(id),       -- NULL = sistem
    snapshot       jsonb,                           -- Undo için gereken
önceki durum
    occurred_at    timestamptz NOT NULL DEFAULT now()
);
```

`report_status_history.snapshot` örneği (teklif seçiminde): `{"selectedOfferId":"…","notSelectedOfferIds":["…","…"]}`. Geri alma işlemi bu kayıttan komutu yeniden kurar (Bölüm 8.6).

6.6 Teklif ve bildirim tabloları

```
CREATE TABLE offers (
    id              uuid PRIMARY KEY,
    report_id       uuid          NOT NULL REFERENCES accident_reports(id),
    mechanic_id     uuid          NOT NULL REFERENCES
mechanic_profiles(user_id),
    total_price     numeric(12,2) NOT NULL CHECK (total_price > 0),
    currency        char(3)       NOT NULL DEFAULT 'TRY',
    estimated_days  smallint      NOT NULL CHECK (estimated_days > 0),
    note            varchar(1000),
    status          varchar(20)   NOT NULL DEFAULT 'Pending'
                    CHECK (status IN
```

```
('Pending','Accepted','NotSelected','Withdrawn','Closed')),
    created_at      timestamptz   NOT NULL DEFAULT now(),
    updated_at      timestamptz,
    decided_at      timestamptz,
    UNIQUE (report_id, mechanic_id)                -- bir usta bir rapora
tek teklif
);

-- Bir raporda yalnızca tek kabul edilmiş teklif olabilir (çift tıklamaya
karşı son savunma hattı)
CREATE UNIQUE INDEX ux_offers_one_accepted_per_report ON offers(report_id)
WHERE status = 'Accepted';

ALTER TABLE accident_reports
    ADD CONSTRAINT fk_accident_reports_selected_offer
    FOREIGN KEY (selected_offer_id) REFERENCES offers(id);

CREATE TABLE offer_price_items (
    id           bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    offer_id     uuid          NOT NULL REFERENCES offers(id) ON DELETE
CASCADE,
    item_type    varchar(20)   NOT NULL CHECK (item_type IN
('Labor','Parts','Paint','Warranty','Other')),
    description  varchar(200),
    amount       numeric(12,2) NOT NULL CHECK (amount >= 0),
    sort_order   smallint      NOT NULL DEFAULT 0
);

CREATE TABLE notifications (
    id          uuid PRIMARY KEY,
    user_id     uuid         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type        varchar(40)  NOT NULL,   -- NEW_JOB, NEW_OFFER,
OFFER_SELECTED, OFFER_NOT_SELECTED ...
    title       varchar(150) NOT NULL,
    body        varchar(500) NOT NULL,
    payload     jsonb,                   -- {"reportId":"…","offerId":"…"} →
tıklayınca ilgili ekrana gider
    is_read     boolean      NOT NULL DEFAULT false,
    created_at  timestamptz  NOT NULL DEFAULT now(),
    read_at     timestamptz
);

CREATE TABLE notification_deliveries (
    id               bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    notification_id  uuid        NOT NULL REFERENCES notifications(id) ON
DELETE CASCADE,
```

```
    channel          varchar(20) NOT NULL CHECK (channel IN
('InApp','Email','Sms')),
    status           varchar(20) NOT NULL CHECK (status IN
('Sent','Failed','Skipped','CircuitOpen')),
    attempt_count    smallint    NOT NULL DEFAULT 1,
    last_error       varchar(500),
    created_at       timestamptz NOT NULL DEFAULT now()
);
```

`notification_deliveries` neden var: Circuit Breaker açıldığında e-posta gönderimi atlanır ama bu sessizce kaybolmamalı. Hangi bildirimin hangi kanaldan gidip gitmediği burada görülür; admin panelinde ve sunumda Circuit Breaker'ın etkisi bu tablo üzerinden gösterilir.

6.7 İndeksler

İndeks	Tablo ve kolonlar	Hangi sorguyu hızlandırır
`ix_mechanic_profiles_matching`	mechanic_profiles(city, approval_status, is_available)	Eşleştirmedeki aday usta sorgusu
`ix_mechanic_specialties_category`	mechanic_specialties(damage_category_id)	Uzmanlık filtresi
`ix_damage_categories_parent`	damage_categories(parent_id)	Ağacı yükleme
`ix_accident_reports_customer`	accident_reports(customer_id, created_at DESC)	"Raporlarım" listesi
`ix_accident_reports_expiry`	accident_reports(expires_at) WHERE status = 'Open'	Süre dolumu arka plan işi
`ix_report_matches_mechanic`	report_matches(mechanic_id, notified_at DESC)	Ustanın "Bana gelen işler" listesi
`ix_offers_report_status`	offers(report_id, status)	Rapor altındaki teklif listesi
`ix_offers_mechanic_status`	offers(mechanic_id, status, created_at DESC)	Ustanın "Tekliflerim" listesi
`ix_notifications_user_unread`	notifications(user_id, is_read, created_at DESC)	Bildirim paneli ve okunmamış sayısı
`ix_report_status_history_report`	report_status_history(report_id, occurred_at DESC)	Geri alma ve durum geçmişi

6.8 Referans veri betiği (`database/reference-data.sql`)

Betik tekrar çalıştırılabilir (`ON CONFLICT DO NOTHING`). İlk migration'dan hemen sonra bir kez çalıştırılır; yeni kategori eklendiğinde yeni satırlar dosyanın sonuna eklenip tekrar çalıştırılır.

```
-- Roller (Identity normalize edilmiş adları büyük harfle arar)
INSERT INTO roles (id, name, normalized_name, concurrency_stamp) VALUES
    (gen_random_uuid(), 'Customer', 'CUSTOMER', gen_random_uuid()::text),
    (gen_random_uuid(), 'Mechanic', 'MECHANIC', gen_random_uuid()::text),
    (gen_random_uuid(), 'Admin',    'ADMIN',    gen_random_uuid()::text)
ON CONFLICT (normalized_name) DO NOTHING;

-- Araç türleri
INSERT INTO vehicle_types (code, name, sort_order) VALUES
    ('CAR','Otomobil',1), ('SUV','SUV / Arazi',2), ('LCV','Hafif ticari',3),
    ('MOTORCYCLE','Motosiklet',4), ('MINIBUS','Minibüs / Panelvan',5)
ON CONFLICT (code) DO NOTHING;

-- Hasar kategorileri: 1. seviye (root_id sonradan kendisine eşitlenir)
INSERT INTO damage_categories (code, name, level, sort_order) VALUES
    ('BODY','Kaporta',1,1), ('PAINT','Boya',1,2),
('MECHANICAL','Mekanik',1,3),
    ('ELECTRICAL','Elektrik',1,4), ('GLASS','Cam',1,5), ('WHEEL','Lastik ve
jant',1,6)
ON CONFLICT (code) DO NOTHING;
UPDATE damage_categories SET root_id = id WHERE level = 1 AND root_id IS
NULL;

-- 2. seviye (örnek: Kaporta altı)
INSERT INTO damage_categories (code, name, level, sort_order, parent_id,
root_id)
SELECT v.code, v.name, 2, v.sort_order, p.id, p.id
FROM (VALUES ('BODY_FRONT_BUMPER','Ön tampon',1), ('BODY_REAR_BUMPER','Arka
tampon',2),
             ('BODY_HOOD','Kaput',3), ('BODY_FENDER','Çamurluk',4),
('BODY_DOOR','Kapı',5),
             ('BODY_TRUNK','Bagaj kapağı',6), ('BODY_ROOF','Tavan',7))
     AS v(code, name, sort_order)
JOIN damage_categories p ON p.code = 'BODY'
ON CONFLICT (code) DO NOTHING;

-- 3. seviye (örnek: Çamurluk altı)
INSERT INTO damage_categories (code, name, level, sort_order, parent_id,
root_id)
SELECT v.code, v.name, 3, v.sort_order, p.id, p.root_id
```

```
FROM (VALUES ('BODY_FENDER_FL','Sol ön çamurluk',1), ('BODY_FENDER_FR','Sağ
ön çamurluk',2),
             ('BODY_FENDER_RL','Sol arka çamurluk',3), ('BODY_FENDER_RR','Sağ
arka çamurluk',4))
     AS v(code, name, sort_order)
JOIN damage_categories p ON p.code = 'BODY_FENDER'
ON CONFLICT (code) DO NOTHING;

-- Sözleşmeler (body metinleri hukuki metin hazır olunca güncellenir)
INSERT INTO legal_documents (code, version, target_role, title, body) VALUES
    ('TERMS_OF_USE','1.0','All','Kullanım Koşulları','...'),
    ('CUSTOMER_KVKK_NOTICE','1.0','Customer','KVKK Aydınlatma Metni','...'),
    ('MECHANIC_SERVICE_AGREEMENT','1.0','Mechanic','Hizmet Sağlayıcı
Sözleşmesi','...')
ON CONFLICT (code, version) DO NOTHING;
```

Diğer 1. seviye kategorilerin alt kırılımları aynı kalıpla eklenir. Admin panelinden eklenen kategoriler de aynı tabloya yazılır; betik yalnızca ilk kurulumu garanti eder.

6.9 İlk admin (`database/admin-bootstrap.sql`)

Önce uygulamadan normal müşteri olarak kaydolunur, sonra aşağıdaki betik e-posta değiştirilerek bir kez çalıştırılır:

```
INSERT INTO user_roles (user_id, role_id)
SELECT u.id, r.id
FROM users u
JOIN roles r ON r.normalized_name = 'ADMIN'
WHERE u.normalized_email = UPPER('admin@ornek.com')
ON CONFLICT DO NOTHING;
```

Kullanıcı bir sonraki girişte yeni JWT aldığında Admin rolüyle yönetim ekranlarını görür.

7. Use case'ler

Projede 4 ana use case ve 1 yönetim akışı var. UC-04 PA2'de uygulanacak use case'tir; diğer üçü PA3'te tamamlanır. Numaralar gereksinimden teste kadar aynen korunur (Bölüm 18).

UC-01: Rol bazlı kayıt ve giriş

- **Aktörler:** Müşteri, Usta
- **Amaç:** Kullanıcının rolüne uygun bilgilerle hesap açması ve rolüne özel arayüze girmesi

- **Ön koşul:** Kullanıcı sisteme kayıtlı değil; referans veri (roller, kategoriler, sözleşmeler) yüklü
- **Ana akış:**
  1. Kullanıcı kayıt ekranında rolünü seçer (Müşteri / Usta).
  2. Sistem role özel formu ve o rolün güncel sözleşmelerini gösterir.
  3. Ortak alanlar girilir: ad soyad, e-posta, telefon, şifre.
  4. Usta ise ek olarak atölye adı, vergi numarası, il, ilçe, adres, konum (haritadan veya cihaz konumundan) ve en az bir uzmanlık seçer.
  5. Kullanıcı sözleşmeleri onaylar ve kaydı gönderir.
  6. Sistem role göre profil fabrikasıyla profili oluşturur, rolü atar, onayları kaydeder.
  7. Müşteri doğrudan giriş yapmış olur ve müşteri paneline yönlenir. Usta "onay bekliyor" ekranını görür.
  8. Sonraki girişlerde sistem JWT ve refresh token üretir; rol bazlı dashboard verisini döner.
- **Alternatif akışlar:** 3a. E-posta zaten kayıtlı → hata. 4a. Vergi numarası zaten kayıtlı → hata. 5a. Zorunlu sözleşme onaylanmadı → gönderim engellenir. 8a. Onayı bekleyen veya reddedilen usta giriş yaparsa yalnızca profil durum ekranını görür, iş listesi kapalıdır. 8b. 5 hatalı şifre denemesi → hesap 15 dakika kilitlenir.
- **Son koşul:** `users`, rol, profil ve `user_consents` kayıtları tek transaction içinde oluşmuştur.
- **Desenler:** Factory Method, Abstract Factory, Proxy (yetkilendirme)

UC-02: Kaza raporu oluşturma

- **Aktör:** Müşteri
- **Amaç:** Hasarı fotoğraf ve yapılandırılmış kriterlerle raporlayıp ustalara açmak
- **Ön koşul:** Müşteri giriş yapmış
- **Ana akış:**
  1. Müşteri "Yeni rapor" sihirbazını açar.
  2. Adım 1: 1–8 fotoğraf yükler. Her fotoğraf yüklendiği anda işlenir (boyutlandırma, sıkıştırma, EXIF/GPS temizleme) ve geçici olarak saklanır.
  3. Adım 2: Hasar ağacında gezinerek bir veya daha fazla hasar bölgesi ve her biri için şiddet seçer.
  4. Adım 3: Araç türünü seçer, isteğe bağlı açıklama yazar.
  5. Adım 4: Konumu cihazdan alır veya il/ilçe seçer.
  6. Adım 5: Özeti görür ve onaylar.

7. Sistem Builder ile raporu kurar, eksik alan varsa reddeder, `Open` durumunda kaydeder, `expires_at` değerini ayardan hesaplar.
8. Kayıt sonrası `AccidentReportCreatedEvent` yayınlanır ve müşteri rapor detay ekranına yönlenir.

- **Alternatif akışlar:** 2a. Desteklenmeyen dosya türü veya boyut sınırı aşımı → o fotoğraf reddedilir. 5a. Konum izni verilmedi → il/ilçe seçimi zorunlu olur. 7a. Aynı müşterinin açık rapor sayısı sınırı (varsayılan 3) aşıldı → hata.
- **Son koşul:** Rapor, hasar kalemleri ve fotoğraflar kaydedilmiş; olay kuyruğa alınmıştır.
- **Desenler:** Builder, Composite, Decorator (görsel hattı), Adapter (dosya depolama)

UC-03: İş bildirimi alma ve teklif verme

- **Aktörler:** Usta (birincil), Sistem (tetikleyen)
- **Amaç:** Uygun ustaların yeni raporu anında öğrenmesi ve kapalı teklif vermesi
- **Ön koşul:** Onaylı ve müsait usta; UC-02 ile açılmış `Open` rapor
- **Ana akış:**
  1. Sistem `AccidentReportCreatedEvent` olayını alır.
  2. Aktif eşleştirme stratejisi (varsayılan şehir bazlı) aday ustaları bulur.
  3. Adaylar filtre zincirinden geçer: onaylı ve aktif → uzmanlık uyuyor → müsait → günlük kapasitesi dolmamış.
  4. Kalan ustalar `report_matches` tablosuna yazılır ve `MechanicsMatchedEvent` yayınlanır.
  5. Her ustaya uygulama içi bildirim düşer (çevrimiçiyse anlık pop-up), e-posta gönderilir.
  6. Usta bildirime tıklar, rapor detayını görür. Diğer ustaların tekliflerini göremez.
  7. Usta fiyat kalemlerini (işçilik, parça, boya, garanti) ve tahmini süreyi girer.
  8. Sistem toplam fiyatı hesaplar, teklifi `Pending` olarak kaydeder, `OfferSubmittedEvent` yayınlar.
  9. Müşterinin ekranına teklif anlık düşer ve bildirim oluşur.
- **Alternatif akışlar:** 2a. Hiç aday yok → müşteriye "şu an bölgenizde uygun usta bulunamadı" bildirimi. 5a. E-posta sağlayıcısı yanıt vermiyor → Circuit Breaker açılır, e-posta `CircuitOpen` olarak kaydedilir, uygulama içi bildirim yine gider. 7a. Usta aynı rapora ikinci teklif vermeye çalışır → mevcut teklifini güncelleme ekranına yönlenir. 8a. Rapor bu arada `OfferSelected` olmuş → 409 ve "rapor artık teklif kabul etmiyor". 8b. Usta `Pending` teklifini geri çekebilir.
- **Son koşul:** Eşleşmeler kaydedilmiş, teklif müşteriye iletilmiştir.
- **Desenler:** Observer / Publish-Subscribe, Strategy, Chain of Responsibility, Decorator (fiyat), Circuit Breaker + Retry, Adapter (bildirim kanalları)

UC-04: Teklif seçimi ve iletişim bilgisinin açılması (PA2)

- **Aktörler:** Müşteri (birincil), Usta (bilgilendirilen)
- **Amaç:** Müşterinin bir teklifi seçmesi, iki tarafın iletişim bilgilerine yalnızca bu anda erişmesi
- **Ön koşul:** Müşterinin `Open` durumda ve en az bir `Pending` teklifi olan raporu var
- **Ana akış:**
  1. Müşteri rapor detayında teklifleri görür ve fiyat, tahmini süre veya en yeniye göre sıralar.
  2. Bir teklifin fiyat kırılımını açar ve "Bu teklifi seç"e basar; onay penceresini onaylar.
  3. Sistem `SelectOfferCommand` oluşturur ve çalıştırır: rapor `OfferSelected`, seçilen teklif `Accepted`, diğerleri `NotSelected` olur; önceki durum `report_status_history.snapshot` alanına yazılır.
  4. `OfferSelectedEvent` yayınlanır; kazanan ustaya "seçildiniz", diğerlerine "başka teklif seçildi" bildirimi gider.
  5. Müşteri ve kazanan usta "eşleşme" ekranında birbirinin ad, telefon ve e-postasını görür; diğer herkes için numara maskelidir.
  6. Müşteri, geri alma süresi boyunca ekranda geri sayım görür.
  7. İş bittiğinde müşteri raporu "tamamlandı" olarak işaretler.
- **Alternatif akışlar:** 3a. Aynı anda iki seçim isteği → ikinci istek kısmi unique index veya `xmin` kontrolüne takılır, 409 döner. 3b. Teklif bu arada geri çekilmiş → 409. 6a. Müşteri süre içinde "vazgeç"e basar → snapshot'tan komut yeniden kurulur, `Undo()` çalışır, rapor `Open`, tüm teklifler `Pending` olur, `OfferSelectionUndoneEvent` yayınlanır. 6b. Süre dolduktan sonra vazgeçme denemesi → 409. 7a. Müşteri iş başlamadan raporu iptal eder → rapor `Cancelled`, teklifler `Closed`.
- **Son koşul:** Raporda tek bir kabul edilmiş teklif vardır; iletişim bilgisi yalnızca eşleşen iki tarafa açıktır.
- **Desenler:** State, Command, Proxy (Protection Proxy), Observer, Strategy (sıralama)

YA-01: Usta onayı ve katalog yönetimi (yönetim akışı)

- **Aktör:** Admin
- **Ana akış:** Admin onay bekleyen ustaları listeler, vergi numarası ve atölye bilgisini kontrol eder, onaylar veya gerekçeyle reddeder. Onaylanan ustaya e-posta gider ve usta eşleştirmeye dahil olur. Admin ayrıca hasar kategorisi ekler, adını değiştirir veya pasifleştirir.
- **Not:** Ödevdeki 4 use case sayısına dahil değildir; üretimde güvenli çalışma için gereklidir. SRS'te "destekleyici akış" olarak yer alır.

8. Application katmanı: servisler ve sözleşmeler

Application katmanında 12 servis interface'i, 11 altyapı interface'i ve 11 repository interface'i bulunur. Controller'lar yalnızca servis interface'lerini, servisler yalnızca altyapı ve repository interface'lerini görür. Her metot `Result` veya `Result<T>` döner; hata durumunda exception yerine `Error(code, message, type)` taşınır ve controller bunu HTTP koduna çevirir.

8.1 Ortak soyutlamalar (`Application/Abstractions`)

```
public interface ICurrentUser
{
    Guid UserId { get; }
    bool IsAuthenticated { get; }
    bool IsInRole(string role);
    string? IpAddress { get; }
}

public interface IDateTimeProvider { DateTimeOffset UtcNow { get; } }

public interface IUnitOfWork
{
    // Kaydeder; başarılıysa biriken domain olaylarını IEventDispatcher'a
iletir
    Task<int> SaveChangesAsync(CancellationToken ct);
}

public interface IEventDispatcher
{
    ValueTask EnqueueAsync(IReadOnlyCollection<IDomainEvent> events,
CancellationToken ct);
}

public interface IEventHandler<in TEvent> where TEvent : IDomainEvent
{
    Task HandleAsync(TEvent domainEvent, CancellationToken ct);
}

public interface IFileStorage
{
    Task SaveAsync(Stream content, string key, string contentType,
CancellationToken ct);
    Task<Stream?> OpenReadAsync(string key, CancellationToken ct);
    Task MoveAsync(string sourceKey, string targetKey, CancellationToken ct);
    Task DeleteAsync(string key, CancellationToken ct);
```

```
}

public interface IImageProcessor
{
    Task<IProcessedImage> ProcessAsync(IImageInput input, CancellationToken
ct);
}

public interface INotificationChannel
{
    NotificationChannelType Channel { get; }
    Task<IDeliveryResult> SendAsync(INotificationMessage message,
CancellationToken ct);
}

public interface IRealtimePublisher
{
    Task PublishToUserAsync(Guid userId, string eventName, object payload,
CancellationToken ct);
}

public interface ITokenService
{
    IAccessToken CreateAccessToken(IUserTokenClaims claims);
    IRefreshTokenPair CreateRefreshToken();   // düz token + SHA-256 özeti
}

public interface IIdentityService
{
    Task<Result<Guid>> CreateUserAsync(ICreateIdentityUserRequest request,
CancellationToken ct);
    Task<Result<IUserTokenClaims>> ValidateCredentialsAsync(ILoginRequest
request, CancellationToken ct);
    Task<IUserTokenClaims?> GetClaimsAsync(Guid userId, CancellationToken
ct);
    Task<IUserContact?> GetContactAsync(Guid userId, CancellationToken ct);
}
```

Neden `IIdentityService` var: `UserManager<ApplicationUser>` Identity kütüphanesine ait ve Infrastructure'da yaşar. Application onu doğrudan kullanırsa Identity'ye bağımlı olur. Bu interface Identity'yi bir port arkasına alır; ileride harici kimlik sağlayıcısına geçmek yalnızca Infrastructure'ı etkiler.

8.2 Repository interface'leri

Interface	Temel metotlar
`ICustomerProfileRepository`	`GetAsync(userId)`, `AddAsync(profile)`
`IMechanicProfileRepository`	`GetAsync(userId)`, `AddAsync`, `ExistsByTaxNumberAsync`, `ListByApprovalStatusAsync`, `FindCandidatesByCityAsync(city, rootCategoryIds)`, `FindCandidatesWithinRadiusAsync(lat, lon, km, rootCategoryIds)`, `CountAcceptedTodayAsync(mechanicIds, day)`
`IDamageCategoryRepository`	`GetTreeAsync(includeInactive)`, `GetByIdsAsync(ids)`, `AddAsync`, `GetAsync(id)`
`IVehicleTypeRepository`	`ListActiveAsync()`, `ExistsAsync(id)`
`ILegalDocumentRepository`	`ListCurrentForRoleAsync(role)`, `GetCurrentByCodesAsync(codes)`
`IAccidentReportRepository`	`AddAsync`, `GetAsync(id)`, `GetWithDetailsAsync(id)`, `GetWithOffersAsync(id)`, `ListByCustomerAsync(customerId, paging)`, `CountOpenByCustomerAsync`, `ListExpiredOpenAsync(now, batchSize)`
`IReportMatchRepository`	`AddRangeAsync`, `IsMatchedAsync(reportId, mechanicId)`, `ListJobsForMechanicAsync(mechanicId, filter, paging)`, `MarkViewedAsync`
`IReportStatusHistoryRepository`	`AddAsync`, `GetLatestAsync(reportId, commandName)`
`IOfferRepository`	`AddAsync`, `GetAsync(id)`, `GetByReportAndMechanicAsync`, `ListForReportAsync(reportId, sort)`, `ListByMechanicAsync(mechanicId, status, paging)`, `ListByReportAsync(reportId)`
`INotificationRepository`	`AddAsync`, `ListByUserAsync(userId, paging)`, `CountUnreadAsync(userId)`, `MarkReadAsync(userId, id)`, `MarkAllReadAsync(userId)`, `AddDeliveryAsync`

Interface	Temel metotlar
`IUserConsentRepository`	`AddRangeAsync`

8.3 Servis interface'leri

Her metodun girdisi bir `I...Request` / `I...Query` interface'idir; WebApi'deki `record`'lar bu interface'leri uygular.

```
// Accounts
public interface IAuthService
{
    Task<Result<RegistrationResponse>> RegisterAsync(IRegisterUserRequest
request, CancellationToken ct);
    Task<Result<AuthResponse>> LoginAsync(ILoginRequest request,
CancellationToken ct);
    Task<Result<AuthResponse>> RefreshAsync(IRefreshTokenRequest request,
CancellationToken ct);
    Task<Result> LogoutAsync(IRefreshTokenRequest request, CancellationToken
ct);
}

public interface IAccountService
{
    Task<Result<MeResponse>> GetMeAsync(CancellationToken ct);
    Task<Result<DashboardResponse>> GetDashboardAsync(CancellationToken ct);
    Task<Result> UpdateCustomerProfileAsync(IUpdateCustomerProfileRequest
request, CancellationToken ct);
    Task<Result> UpdateMechanicProfileAsync(IUpdateMechanicProfileRequest
request, CancellationToken ct);
    Task<Result> SetAvailabilityAsync(ISetAvailabilityRequest request,
CancellationToken ct);
}

// Catalog
public interface ICatalogService
{
    Task<IReadOnlyList<DamageCategoryNodeResponse>>
GetDamageTreeAsync(CancellationToken ct);
    Task<IReadOnlyList<VehicleTypeResponse>>
GetVehicleTypesAsync(CancellationToken ct);
    Task<IReadOnlyList<LegalDocumentResponse>>
GetLegalDocumentsAsync(IRoleQuery query, CancellationToken ct);
}
```

```
// Reports
public interface IReportService
{
    Task<Result<ReportCreatedResponse>> CreateAsync(ICreateReportRequest
request, CancellationToken ct);
    Task<Result<CustomerReportDetailResponse>> GetForCustomerAsync(Guid
reportId, CancellationToken ct);
    Task<Result<PagedResult<ReportListItemResponse>>>
ListMineAsync(IPagedQuery query, CancellationToken ct);
    Task<Result> CancelAsync(Guid reportId, CancellationToken ct);
    Task<Result> CompleteAsync(Guid reportId, CancellationToken ct);
    Task<Result<MechanicJobDetailResponse>> GetForMechanicAsync(Guid
reportId, CancellationToken ct);
    Task<Result<PagedResult<MechanicJobListItemResponse>>>
ListMechanicJobsAsync(IMechanicJobsQuery query, CancellationToken ct);
}

public interface IPhotoService
{
    Task<Result<TemporaryPhotoResponse>>
UploadTemporaryAsync(IUploadPhotoRequest request, CancellationToken ct);
    Task<Result<IPhotoStream>> OpenAsync(Guid photoId, CancellationToken ct);
// yetki: sahip, eşleşen usta, admin
}

// Matching (yalnızca olay handler'ı çağırır, controller'ı yok)
public interface IMatchingService
{
    Task<Result<MatchingResult>> MatchAsync(Guid reportId, CancellationToken
ct);
}

// Offers
public interface IOfferService
{
    Task<Result<OfferResponse>> SubmitAsync(ISubmitOfferRequest request,
CancellationToken ct);
    Task<Result<OfferResponse>> UpdateAsync(IUpdateOfferRequest request,
CancellationToken ct);
    Task<Result> WithdrawAsync(Guid offerId, CancellationToken ct);
    Task<Result<IReadOnlyList<OfferListItemResponse>>>
ListForReportAsync(IReportOffersQuery query, CancellationToken ct); //
müşteri
    Task<Result<OfferResponse?>> GetMyOfferForReportAsync(Guid reportId,
CancellationToken ct);                          // usta
    Task<Result<PagedResult<MyOfferListItemResponse>>>
```

```
ListMineAsync(IMyOffersQuery query, CancellationToken ct);       // usta
}

public interface IOfferSelectionService
{
    Task<Result<SelectionResponse>> SelectAsync(ISelectOfferRequest request,
CancellationToken ct);
    Task<Result> UndoAsync(IUndoSelectionRequest request, CancellationToken
ct);
    Task<Result<SelectionResponse>> GetAsync(Guid reportId, CancellationToken
ct);  // kalan geri alma süresi dahil
}

// Contacts
public interface IContactService
{
    Task<Result<MatchContactResponse>> GetMatchContactAsync(Guid reportId,
CancellationToken ct);
}

// Notifications
public interface INotificationService
{
    Task NotifyAsync(INotifyRequest request, CancellationToken ct);   //
yalnızca olay handler'ları çağırır
    Task<Result<PagedResult<NotificationResponse>>> ListMineAsync(IPagedQuery
query, CancellationToken ct);
    Task<int> GetUnreadCountAsync(CancellationToken ct);
    Task<Result> MarkReadAsync(Guid notificationId, CancellationToken ct);
    Task<Result> MarkAllReadAsync(CancellationToken ct);
}

// Admin
public interface IMechanicApprovalService
{
    Task<Result<PagedResult<PendingMechanicResponse>>>
ListAsync(IMechanicApprovalQuery query, CancellationToken ct);
    Task<Result> ApproveAsync(Guid mechanicId, CancellationToken ct);
    Task<Result> RejectAsync(IRejectMechanicRequest request,
CancellationToken ct);
}

public interface ICategoryAdminService
{
    Task<Result<DamageCategoryNodeResponse>>
CreateAsync(ICreateCategoryRequest request, CancellationToken ct);
```

```
    Task<Result> UpdateAsync(IUpdateCategoryRequest request,
CancellationToken ct);
    Task<Result> SetActiveAsync(ISetCategoryActiveRequest request,
CancellationToken ct);
}
```

8.4 Request sözleşmeleri

Interface	Alanlar	Uygulayan record (WebApi)
`IRegisterUserRequest`	Role, FullName, Email, Phone, Password, AcceptedLegalDocumentIds	taban sözleşme
`IRegisterCustomerRequest : IRegisterUserRequest`	City, District	`RegisterCustomerRequest`
`IRegisterMechanicRequest : IRegisterUserRequest`	WorkshopName, TaxNumber, City, District, Address, Latitude, Longitude, SpecialtyCategoryIds	`RegisterMechanicRequest`
`ILoginRequest`	Email, Password	`LoginRequest`
`IRefreshTokenRequest`	RefreshToken	`RefreshTokenRequest`
`ISetAvailabilityRequest`	IsAvailable	`SetAvailabilityRequest`
`IUploadPhotoRequest`	Content (Stream), FileName, ContentType, Length	`UploadPhotoRequest` (IFormFile'dan)
`ICreateReportRequest`	VehicleTypeId, Description, City, District, AddressText, Latitude, Longitude, Damages[{CategoryId, Severity, Note}], TemporaryPhotoKeys[]	`CreateReportRequest`
`IPagedQuery`	Page, PageSize	`PagedQuery`
`IMechanicJobsQuery : IPagedQuery`	Filter (New / Offered / Won / Lost)	`MechanicJobsQuery`
`ISubmitOfferRequest`	ReportId, EstimatedDays, Note, PriceItems[{ItemType, Description, Amount}]	`SubmitOfferRequest`

Interface	Alanlar	Uygulayan record (WebApi)
`IUpdateOfferRequest : ISubmitOfferRequest`	OfferId	`UpdateOfferRequest`
`IReportOffersQuery`	ReportId, SortKey (price / newest / fastest)	`ReportOffersQuery`
`ISelectOfferRequest`	ReportId, OfferId	`SelectOfferRequest`
`IUndoSelectionRequest`	ReportId	`UndoSelectionRequest`
`INotifyRequest`	UserId, Type, Title, Body, Payload, Channels	`NotifyRequest` (Application içinde)
`IRejectMechanicRequest`	MechanicId, Reason	`RejectMechanicRequest`
`ICreateCategoryRequest`	ParentId, Code, Name, SortOrder	`CreateCategoryRequest`

FluentValidation validator'ları interface üzerine yazılır: `class SubmitOfferValidator : AbstractValidator<ISubmitOfferRequest>`. Örnek kurallar: e-posta formatı, telefon `^05\d{9}$`, vergi numarası 10 veya 11 hane, şifre en az 8 karakter ve rakam içerir, fiyat kalemi en az 1 ve toplam > 0, fotoğraf anahtarı sayısı 1 ile `Photos:MaxCountPerReport` arası. Açık rapor sınırı `Reports:MaxOpenPerCustomer` ayarından okunur (varsayılan 3).

8.5 Desen arayüzleri

```
// Factory Method: rol başına profil üretimi (UC-01)
public interface IUserProfileFactory
{
    string Role { get; }
    Task<Result> CreateProfileAsync(Guid userId, IRegisterUserRequest
request, CancellationToken ct);
    IReadOnlyCollection<string> RequiredLegalDocumentCodes { get; }
}

// Abstract Factory: rol başına tutarlı panel ailesi (UC-01)
public interface IDashboardFactory
{
    string Role { get; }
    IReadOnlyList<MenuItemResponse> CreateMenu();
    Task<DashboardSummaryResponse> CreateSummaryAsync(Guid userId,
CancellationToken ct);
    IReadOnlyList<QuickActionResponse> CreateQuickActions();
}
```

```
// Strategy: aday usta bulma (UC-03)
public interface IMatchingStrategy
{
    string Code { get; }   // CITY, RADIUS
    Task<IReadOnlyList<MechanicCandidate>>
FindCandidatesAsync(IMatchingContext context, CancellationToken ct);
}

// Chain of Responsibility: aday filtreleme (UC-03)
public interface IMechanicFilter
{
    IMechanicFilter SetNext(IMechanicFilter next);
    Task<IReadOnlyList<MechanicCandidate>> ApplyAsync(IMatchingContext
context,
        IReadOnlyList<MechanicCandidate> candidates, CancellationToken ct);
}

// Decorator: fiyat kırılımı (UC-03, Domain/Offers/Pricing)
public interface IPriceComponent
{
    decimal Total { get; }
    IReadOnlyList<PriceLine> Lines { get; }
}

// Strategy: teklif sıralama (UC-04)
public interface IOfferSortStrategy
{
    string Key { get; }    // price, newest, fastest
    IQueryable<Offer> Apply(IQueryable<Offer> offers);
}

// Command: geri alınabilir rapor işlemleri (UC-04)
public interface IReportCommand
{
    string Name { get; }
    void Execute();
    void Undo();
    ReportCommandSnapshot CreateSnapshot();
}

public interface IReportCommandFactory
{
    IReportCommand CreateSelectOffer(AccidentReport report, Offer offer, Guid
actorId, DateTimeOffset now);
    IReportCommand Restore(AccidentReport report, ReportStatusHistory entry,
IReadOnlyList<Offer> offers,
```

```
        Guid actorId, DateTimeOffset now);
}

// Proxy: iletişim bilgisine kontrollü erişim (UC-04, Domain/Contacts)
public interface IContactInfo
{
    string FullName { get; }
    string Phone { get; }
    string Email { get; }
    bool IsRevealed { get; }
}
```

8.6 Geri alma (Undo) nasıl çalışır

HTTP istekleri durumsuzdur; seçim yapan istekteki komut nesnesi, geri alma isteği geldiğinde bellekte yoktur. Bu yüzden komut kendini kalıcı hale getirir:

1. `SelectOfferCommand.Execute()` çalışmadan önce `CreateSnapshot()` ile seçilen teklifin ve `NotSelected` yapılacak tekliflerin kimliklerini alır.
2. `OfferSelectionService` bu snapshot'ı `report_status_history` tablosuna `command_name = 'SelectOfferCommand'` ile yazar.
3. Geri alma isteğinde servis en son `SelectOfferCommand` kaydını okur, `IReportCommandFactory.Restore(...)` ile aynı komutu yeniden kurar ve `Undo()` çağırır.
4. `Undo()` raporu State üzerinden `Open`'a döndürür (süre kontrolü State'te), teklifleri `Pending` yapar, geçmişe `command_name = 'UndoSelectOffer'` kaydı ekler.

Bu yaklaşım Command deseninin "işlemi nesne olarak saklama" niyetini veritabanına taşır; aynı zamanda denetim kaydı da sağlar.

8.7 Geçici fotoğraf akışı

Fotoğraflar rapor oluşmadan yüklenir, çünkü sihirbazın ilk adımı fotoğraftır ve kullanıcı her fotoğrafın işlendiğini hemen görmelidir.

1. `IPhotoService.UploadTemporaryAsync` fotoğrafı işler ve `temp/{userId}/{guid}.jpg` anahtarıyla saklar; anahtarı döner.
2. `IReportService.CreateAsync` gelen anahtarların `temp/{currentUserId}/` önekiyle başladığını doğrular. Başka kullanıcının dosyasını rapora bağlamak böylece imkânsızdır.
3. Rapor kaydedilirken dosyalar `reports/{yyyy}/{MM}/{reportId}/{photoId}.jpg` konumuna taşınır ve `report_photos` satırları yazılır.
4. `TemporaryUploadCleanupJob` 24 saatten eski geçici dosyaları siler.

Risk notu: Rapor kaydı başarılı olup dosya taşıma yarıda kalırsa kayıt ile disk tutarsız kalabilir. Önce taşıma, sonra kayıt yapılır; kayıt başarısız olursa taşınan dosyalar silinir. Tam tutarlılık için Future Work'te Outbox önerilir.

9. Domain olayları ve handler'lar

Sistemde 12 domain olayı ve bunları dinleyen 11 handler vardır. Olay yayınlamak için hiçbir servis başka bir modülün servisini çağırmaz; olayı entity'nin içinde üretir, gerisini altyapı taşır.

9.1 Olayın yolculuğu

1. Domain metodu (örneğin `AccidentReport.SelectOffer`) olayı entity'nin `DomainEvents` listesine ekler.
2. `AppDbContext.SaveChangesAsync`, değişen entity'lerden olayları toplar, kaydı yapar ve yalnızca kayıt başarılıysa `IEventDispatcher.EnqueueAsync` çağırır.
3. `InProcessEventDispatcher` olayları bellekteki `System.Threading.Channels` kuyruğuna yazar ve hemen döner. HTTP yanıtı eşleştirmeyi veya e-postayı beklemez.
4. `EventProcessingBackgroundService` kuyruğu okur, her olay için yeni bir DI scope açar ve o olay tipinin tüm `IEventHandler<T>` implementasyonlarını çalıştırır.
5. Bir handler hata verirse loglanır ve diğer handler'lar çalışmaya devam eder; bir bildirim hatası eşleştirmeyi durdurmaz.

Risk notu (geçici çözüm): Kuyruk bellekte olduğu için uygulama tam olay kuyruğa yazıldıktan sonra kapanırsa olay kaybolur. Tek sunuculu ilk sürüm için kabul edilebilir; kalıcı çözüm Transactional Outbox'tır (Bölüm 20).

9.2 Olaylar, içerikleri ve handler'ları

Olay	Taşıdığı alanlar	Handler	Ne yapar
`AccidentReportCreatedEvent`	ReportId, CustomerId, City, Latitude, Longitude, RootCategoryIds	`AccidentReportCreatedMatchingHandler`	`IMatchingService.MatchAsync` çağırır
`MechanicsMatchedEvent`	ReportId, CustomerId, MechanicIds	`MechanicsMatchedNotificationHandler`	Her ustaya `NEW_JOB` (uygulama içi + e-posta); liste boşsa müşteriye `NO_MECHANIC_FOUND`
`OfferSubmittedEvent`	OfferId, ReportId, MechanicId, CustomerId, TotalPrice	`OfferSubmittedNotificationHandler`	Müşteriye `NEW_OFFER`
`OfferSubmittedEvent`	(aynı)	`OfferChangedRealtimeHandler`	Müşterinin açık rapor ekranına `offer.created`
`OfferUpdatedEvent`	OfferId, ReportId, CustomerId, TotalPrice	`OfferChangedRealtimeHandler`	Müşteriye `offer.updated`

Olay	Taşıdığı alanlar	Handler	Ne yapar
`OfferWithdrawnEvent`	OfferId, ReportId, CustomerId	`OfferChangedRealtimeHandler`	Müşteriye `offer.withdrawn`
`OfferSelectedEvent`	ReportId, SelectedOfferId, WinnerMechanicId, OtherMechanicIds, CustomerId	`OfferSelectedNotificationHandler`	Kazanana `OFFER_SELECTED`, diğerlerine `OFFER_NOT_SELECTED`
`OfferSelectionUndoneEvent`	ReportId, MechanicIds	`OfferSelectionUndoneNotificationHandler`	Tüm teklif veren ustalara `SELECTION_UNDONE`
`ReportCompletedEvent`	ReportId, WinnerMechanicId	`ReportCompletedNotificationHandler`	Ustaya `JOB_COMPLETED`
`ReportCancelledEvent`	ReportId, MechanicIdsWithOffers	`ReportCancelledNotificationHandler`	Ustalara `REPORT_CANCELLED`
`ReportExpiredEvent`	ReportId, CustomerId	`ReportExpiredNotificationHandler`	Müşteriye `REPORT_EXPIRED`
`MechanicApprovedEvent` / `MechanicRejectedEvent`	MechanicId, Reason	`MechanicApprovalNotificationHandler`	Ustaya e-posta ve uygulama içi bildirim

Seçim, iptal ve süre dolumu gibi durum değişikliklerinde ayrıca `ReportStatusRealtimeHandler`, ilgili tüm kullanıcılara `report.statusChanged` gönderir; açık ekranlar sayfa yenilenmeden güncellenir.

9.3 Bildirim hattı

`INotificationService.NotifyAsync` her bildirim için aynı sırayı izler:

1. `notifications` tablosuna kaydeder. Kullanıcı çevrimdışıysa bile bildirim kaybolmaz.
2. `IRealtimePublisher` ile `notification.created` olayını gönderir (kullanıcı çevrimiçiyse pop-up).
3. İstenen ek kanallar için (`Email`, ileride `Sms`) ilgili `INotificationChannel`'ı çağırır. Bu kanallar `ResilientNotificationChannel` ile sarılıdır: Retry + Circuit Breaker.
4. Her kanalın sonucunu `notification_deliveries` tablosuna yazar (`Sent`, `Failed`, `CircuitOpen`, `Skipped`).

10. Desen eşleme matrisi

16 farklı desen 18 yerde uygulanıyor: 16'sı bir use case'e bağlı, 2'si tüm sistemde geçerli. Her desenin kodu (PAT-xx.y) koddaki yorumda, SDD'de ve testte aynen geçer.

Kod	Desen	UC	Sınıflar	Neden burada	Olmasaydı	Ölçülebilir metrik
PAT-01.1	Factory Method	UC-01	`IUserProfileFactory`, `CustomerProfileFactory`, `MechanicProfileFactory`	Tek `RegisterAsync` metodu role göre doğru profili üretir	`AuthService` içinde rol başına `if/switch` büyür	Yeni rol eklerken değişen mevcut sınıf: 0

Kod	Desen	UC	Sınıflar	Neden burada	Olmasaydı	Ölçülebilir metrik
PAT-01.2	Abstract Factory	UC-01	`IDashboardFactory`, `Customer/Mechanic/AdminDashboardFactory`	Menü, özet ve hızlı işlemler aynı rol ailesinden gelir	Ustaya müşteri menüsü karışabilir	Rol başına tek fabrika; yanlış rol-panel eşleşmesi testte 0
PAT-02.1	Builder	UC-02	`AccidentReportBuilder`	Adım adım toplanan veriden geçerli rapor kurar	Yarım rapor veritabanına yazılabilir	Eksik alanlı rapor oluşturma testi: %100 red
PAT-02.2	Composite	UC-02	`IDamageComponent`, `DamageCategory`	3 seviyeli hasar ağacı tek arayüzle gezilir	Seviye başına ayrı kod	Yeni kategori = yalnızca veri, kod değişikliği 0
PAT-02.3	Decorator	UC-02	`ResizeProcessor`, `CompressionProcessor`, `ExifStripProcessor`	Görsel işleme adımları sırayla eklenir, çıkarılır	Tek dev metot	Ortalama dosya boyutunda % azalma; çıktı dosyada GPS etiketi: 0
PAT-02.4	Adapter	UC-02	`IFileStorage` ← `LocalFileStorage`	Depolama teknolojisi use case'ten gizlenir	S3'e geçişte servisler değişir	Depolama değişince değişen Application satırı: 0
PAT-03.1	Observer / Publish-Subscribe	UC-03	`IEventDispatcher`, `IEventHandler<T>`, `InProcessEventDispatcher`	Rapor modülü eşleştirmeyi bilmez	Servisler zincirleme çağrılır, sıkı bağlılık	Rapor oluşturma yanıt süresi eşleştirmeden bağımsız (p95 < 500 ms)
PAT-03.2	Strategy	UC-03	`IMatchingStrategy`, `CityMatchingStrategy`, `RadiusMatchingStrategy`	Aday bulma yöntemi ayardan değişir	Şehir/mesafe `if`'leri servis içinde	Yeni strateji eklerken değişen sınıf: 0
PAT-03.3	Chain of Responsibility (+ Template Method)	UC-03	`IMechanicFilter`, `MechanicFilterBase`, 4 filtre	Filtreler bağımsız, sırası değiştirilebilir; taban sınıf "filtrele, sonra devret" iskeletini sabitler	Tek sorguda iç içe koşullar	Filtre ekleme mevcut filtre testlerini etkilemez
PAT-03.4	Decorator	UC-03	`IPriceComponent`, `BasePrice`, `PartsCost`, `PaintCost`, `WarrantyCost`	Fiyat kalemleri üst üste eklenir, kırılım korunur	Fiyat tek sayı, müşteri kırılımı göremez	Toplam = kalemler toplamı (property test)
PAT-03.5	Adapter	UC-03	`INotificationChannel` ← `SmtpEmailChannel`, `LoggingSmsChannel`	Kanal kütüphaneleri (MailKit vb.) arkaya alınır	Servisler SMTP bilir	SMS sağlayıcısı eklemek = 1 sınıf
PAT-03.6	Circuit Breaker + Retry	UC-03	`ResilientNotificationChannel` (Polly)	E-posta sağlayıcısı çökünce sistem beklemez	Her istek zaman aşımına kadar bekler	Sağlayıcı kapalıyken bildirim işleme süresi sabit; `CircuitOpen` kayıt sayısı
PAT-04.1	State	UC-04	`IReportState`, 5 durum sınıfı, `ReportStateFactory`	Her durumda izinli işlemler kendi sınıfında	`if (status == ...)` kontrolleri kod geneline dağılır	Geçersiz geçiş testleri %100 red
PAT-04.2	Command	UC-04	`IReportCommand`, `SelectOfferCommand`, `IReportCommandFactory`	Çok adımlı seçim tek nesne, snapshot ile geri alınabilir	Geri alma için ayrı, kopyalanmış mantık	Undo sonrası durum = seçim öncesi durum (test)
PAT-04.3	Proxy (Protection)	UC-04	`IContactInfo`, `ContactInfo`, `ContactInfoProxy`	Numara gösterme kararı tek yerde	Her ekran kendi kontrolünü yapar, unutulabilir	Yetkisiz yanıtta gerçek numara: 0

Kod	Desen	UC	Sınıflar	Neden burada	Olmasaydı	Ölçülebilir metrik
PAT-04.4	Strategy	UC-04	`IOfferSortStrategy` + 3 sıralama	Sıralama anahtarı istekten seçilir	Controller'da `switch`	Yeni sıralama = 1 sınıf
PAT-X.1	Repository + Unit of Work	Tümü	`I*Repository`, `IUnitOfWork`	Use case'ler EF Core'u bilmez	Servisler DbContext'e bağlı	Unit testlerin veritabanısız çalışma oranı: %100
PAT-X.2	Singleton (DI container)	Tümü	`IDateTimeProvider`, `EventQueue`, strateji çözümleyicileri	Tekil nesneleri container yönetir	Elle `getInstance()`: test edilemez, global durum	Kodda statik `Instance` alanı: 0

Proxy ile Decorator farkı (sunum için öğretici nokta): İkisi de aynı interface'i uygulayıp bir iç nesneyi sarar. `ContactInfoProxy` erişimi kontrol eder, davranış eklemez. `PriceComponentDecorator` davranış ekler, erişimi kısıtlamaz. `ResilientNotificationChannel` yapısal olarak bir Decorator'dır; niyeti Circuit Breaker'dır.

10.1 Kod yorumu şablonu

Her desenin uygulandığı sınıfın başına aşağıdaki blok eklenir (PA2 zorunluluğu):

```
// [UC-04][PAT-04.2] Command
// Niyet     : "Teklif seç" işlemini (rapor durumu + teklif durumları) tek bir nesnede toplamak.
// Neden     : Seçim birden çok değişiklik içerir ve süre içinde geri alınabilmelidir.
// Kazanım   : Execute/Undo simetrik; geri alma mantığı kopyalanmaz, snapshot ile denetim kaydı oluşur.
// Metrik    : Undo sonrası rapor ve teklif durumları seçim öncesiyle birebir aynı (SelectOfferCommandTests).
```

10.2 Proje kartı (dersin formatı)

Alan	İçerik
Konu	IndustryApp: Kaza sonrası onarım teklif platformu
Birincil mimari örüntü	Layered (Clean Architecture) modüler monolit; modüller arası Publish-Subscribe; dağıtık MVC
Hibrit örüntüler	Observer + Strategy + Chain of Responsibility + State + Command + Proxy + Circuit Breaker (+ Builder, Factory Method, Abstract Factory, Composite, Decorator, Adapter)

Alan	İçerik
Senaryo	Müşteri hasarı fotoğraf ve kriterlerle raporlar; sistem olay tabanlı olarak uygun ustaları bulur; ustalar kapalı teklif verir; müşteri seçer; iletişim bilgisi yalnız eşleşme anında açılır ve seçim süre içinde geri alınabilir
Öğretici nokta	Strategy (adayı nasıl bulurum) ile Chain of Responsibility (hangi filtreden kim geçer) farkı; State + Command ile geri alınabilir durum geçişi; Proxy ile Decorator'ın aynı yapı, farklı niyeti; Pub-Sub iş akışını, Circuit Breaker sistem sağlığını yönetir
Güncel eğilim ve etik	Esnafın dijitalleşmesi ve ters açık artırma pazaryerleri. Kapalı teklifin fiyat adaleti, KVKK ve iletişim bilgisi gizliliği, fotoğraflardaki konum/plaka verisi, sahte hasar fotoğrafıyla sigorta dolandırıcılığı riski, usta onayında dijital erişim eşitsizliği

11. WebApi: controller'lar ve endpoint'ler

WebApi'de 12 controller ve 42 endpoint vardır (41. satır iki endpoint içerir). Her controller tek bir kaynaktan sorumludur, yalnızca kendi servis interface'ini çağırır ve 10–15 satırı geçmeyen metotlardan oluşur.

11.1 Controller kuralları

- Tüm controller'lar `ApiControllerBase`'den türer. Bu taban sınıfta yalnızca `ToActionResult(Result)` dönüşümü vardır; iş mantığı yoktur.
- Controller metodu şu üç şeyi yapar: request record'unu alır, servis interface metodunu çağırır, `Result`'ı HTTP yanıtına çevirir.
- Controller'lar birbirini çağırmaz, `DbContext` veya repository kullanmaz, Infrastructure sınıfı görmez.
- Yetki, attribute ile controller veya metot seviyesinde tanımlanır; kaynak sahipliği (bu rapor bu müşterinin mi) serviste kontrol edilir.

11.2 Yetki politikaları

Politika	Kural
`Authenticated`	Geçerli JWT
`CustomerOnly`	Rol: Customer
`ApprovedMechanic`	Rol: Mechanic ve token'da `mechanic_status = Approved` claim'i
`MechanicAny`	Rol: Mechanic (onay durumundan bağımsız; profil ve durum ekranı için)
`AdminOnly`	Rol: Admin

11.3 Endpoint listesi

#	Controller	Metot ve yol	Yetki	Servis çağrısı	UC
1	`AuthController`	`POST /api/auth/register/customer`	Anonim	`IAuthService.RegisterAsync`	UC-01
2	`AuthController`	`POST /api/auth/register/mechanic`	Anonim	`IAuthService.RegisterAsync`	UC-01
3	`AuthController`	`POST /api/auth/login`	Anonim	`IAuthService.LoginAsync`	UC-01
4	`AuthController`	`POST /api/auth/refresh`	Anonim	`IAuthService.RefreshAsync`	UC-01
5	`AuthController`	`POST /api/auth/logout`	Authenticated	`IAuthService.LogoutAsync`	UC-01
6	`AccountController`	`GET /api/account/me`	Authenticated	`IAccountService.GetMeAsync`	UC-01
7	`AccountController`	`GET /api/account/dashboard`	Authenticated	`IAccountService.GetDashboardAsync`	UC-01
8	`AccountController`	`PUT /api/account/customer-profile`	CustomerOnly	`IAccountService.UpdateCustomerProfileAsync`	UC-01
9	`AccountController`	`PUT /api/account/mechanic-profile`	MechanicAny	`IAccountService.UpdateMechanicProfileAsync`	UC-01
10	`AccountController`	`PUT /api/account/availability`	ApprovedMechanic	`IAccountService.SetAvailabilityAsync`	UC-03
11	`CatalogController`	`GET /api/catalog/damage-categories`	Anonim	`ICatalogService.GetDamageTreeAsync`	UC-01, UC-02
12	`CatalogController`	`GET /api/catalog/vehicle-types`	Anonim	`ICatalogService.GetVehicleTypesAsync`	UC-02
13	`CatalogController`	`GET /api/catalog/legal-documents?role=`	Anonim	`ICatalogService.GetLegalDocumentsAsync`	UC-01
14	`PhotosController`	`POST /api/photos/temporary` (multipart)	CustomerOnly	`IPhotoService.UploadTemporaryAsync`	UC-02
15	`PhotosController`	`GET /api/photos/{photoId}`	Authenticated	`IPhotoService.OpenAsync`	UC-02, UC-03
16	`ReportsController`	`POST /api/reports`	CustomerOnly	`IReportService.CreateAsync`	UC-02
17	`ReportsController`	`GET /api/reports?page=&pageSize=`	CustomerOnly	`IReportService.ListMineAsync`	UC-02
18	`ReportsController`	`GET /api/reports/{reportId}`	CustomerOnly	`IReportService.GetForCustomerAsync`	UC-02, UC-04

#	Controller	Metot ve yol	Yetki	Servis çağrısı	UC
19	`ReportsController`	`POST /api/reports/{reportId}/cancel`	CustomerOnly	`IReportService.CancelAsync`	UC-04
20	`ReportsController`	`POST /api/reports/{reportId}/complete`	CustomerOnly	`IReportService.CompleteAsync`	UC-04
21	`MechanicJobsController`	`GET /api/mechanic/jobs?filter=&page=`	ApprovedMechanic	`IReportService.ListMechanicJobsAsync`	UC-03
22	`MechanicJobsController`	`GET /api/mechanic/jobs/{reportId}`	ApprovedMechanic	`IReportService.GetForMechanicAsync`	UC-03
23	`OffersController`	`POST /api/offers`	ApprovedMechanic	`IOfferService.SubmitAsync`	UC-03
24	`OffersController`	`PUT /api/offers/{offerId}`	ApprovedMechanic	`IOfferService.UpdateAsync`	UC-03
25	`OffersController`	`POST /api/offers/{offerId}/withdraw`	ApprovedMechanic	`IOfferService.WithdrawAsync`	UC-03
26	`OffersController`	`GET /api/offers/mine?status=&page=`	ApprovedMechanic	`IOfferService.ListMineAsync`	UC-03
27	`OffersController`	`GET /api/reports/{reportId}/offers?sort=`	CustomerOnly	`IOfferService.ListForReportAsync`	UC-04
28	`OffersController`	`GET /api/mechanic/jobs/{reportId}/my-offer`	ApprovedMechanic	`IOfferService.GetMyOfferForReportAsync`	UC-03
29	`OfferSelectionController`	`POST /api/reports/{reportId}/selection`	CustomerOnly	`IOfferSelectionService.SelectAsync`	UC-04
30	`OfferSelectionController`	`DELETE /api/reports/{reportId}/selection`	CustomerOnly	`IOfferSelectionService.UndoAsync`	UC-04
31	`OfferSelectionController`	`GET /api/reports/{reportId}/selection`	Authenticated	`IOfferSelectionService.GetAsync`	UC-04
32	`ContactsController`	`GET /api/reports/{reportId}/contact`	Authenticated	`IContactService.GetMatchContactAsync`	UC-04
33	`NotificationsController`	`GET /api/notifications?page=`	Authenticated	`INotificationService.ListMineAsync`	Tümü
34	`NotificationsController`	`GET /api/notifications/unread-count`	Authenticated	`INotificationService.GetUnreadCountAsync`	Tümü
35	`NotificationsController`	`POST /api/notifications/{id}/read`	Authenticated	`INotificationService.MarkReadAsync`	Tümü
36	`NotificationsController`	`POST /api/notifications/read-all`	Authenticated	`INotificationService.MarkAllReadAsync`	Tümü
37	`AdminMechanicsController`	`GET /api/admin/mechanics?status=&page=`	AdminOnly	`IMechanicApprovalService.ListAsync`	YA-01
38	`AdminMechanicsController`	`POST /api/admin/mechanics/{id}/approve`	AdminOnly	`IMechanicApprovalService.ApproveAsync`	YA-01
39	`AdminMechanicsController`	`POST /api/admin/mechanics/{id}/reject`	AdminOnly	`IMechanicApprovalService.RejectAsync`	YA-01
40	`AdminCatalogController`	`POST /api/admin/damage-categories`	AdminOnly	`ICategoryAdminService.CreateAsync`	YA-01
41	`AdminCatalogController`	`PUT /api/admin/damage-categories/{id}` ve `POST .../{id}/active`	AdminOnly	`ICategoryAdminService.UpdateAsync`, `SetActiveAsync`	YA-01

Kapalı teklif kuralı nerede uygulanır: Usta tarafındaki hiçbir endpoint (21, 22, 26, 28) başka ustanın teklifini döndürmez. `MechanicJobDetailResponse` modelinde teklif listesi alanı hiç yoktur, yalnızca `MyOffer` alanı vardır. Bu yüzden kural bir `if` ile değil, modelin şekliyle garanti edilir.

11.4 Hata formatı

Tüm hatalar RFC 7807 `ProblemDetails` biçiminde döner ve makinece okunabilir bir `code` alanı taşır. Frontend kullanıcıya gösterilecek Türkçe mesajı bu koda göre seçer.

```
{
  "type": "https://industryapp.com/errors/report-invalid-state",
  "title": "Rapor bu işlem için uygun durumda değil.",
  "status": 409,
  "code": "REPORT_INVALID_STATE",
  "traceId": "00-4bf92f35...",
  "errors": null
}
```

`Error.Type`	HTTP	Örnek kod
Validation	400	`VALIDATION_FAILED` (alan hataları `errors` içinde)
Unauthorized	401	`INVALID_CREDENTIALS`, `TOKEN_EXPIRED`
Forbidden	403	`NOT_REPORT_OWNER`, `NOT_MATCHED`, `MECHANIC_NOT_APPROVED`
NotFound	404	`REPORT_NOT_FOUND`, `OFFER_NOT_FOUND`
Conflict	409	`REPORT_INVALID_STATE`, `OFFER_ALREADY_EXISTS`, `UNDO_WINDOW_EXPIRED`, `CONCURRENCY_CONFLICT`
BusinessRule	422	`OPEN_REPORT_LIMIT_REACHED`, `EMAIL_ALREADY_USED`, `TAX_NUMBER_ALREADY_USED`

Beklenmeyen exception'lar `GlobalExceptionHandler` (.NET 8 `IExceptionHandler`) tarafından yakalanır, `traceId` ile loglanır ve kullanıcıya iç ayrıntı sızdırmadan 500 döner. PostgreSQL unique ihlali (`23505`) `CONFLICT` hatasına çevrilir.

12. SignalR: gerçek zamanlı iletişim

Tek bir hub (`/hubs/notifications`) vardır ve yalnızca sunucudan istemciye mesaj gönderir. İstemci hub üzerinden işlem yapmaz; tüm işlemler REST endpoint'lerinden geçer. Böylece yetki ve doğrulama tek yerde kalır.

- **Kimlik:** Hub `[Authorize]` ile korunur. WebSocket başlık gönderemediği için JWT, bağlantı sırasında `access_token` sorgu parametresiyle gelir; JwtBearer'ın `OnMessageReceived` olayı yalnızca `/hubs` yolu için bu parametreyi okur.
- **Hedefleme:** Mesajlar `Clients.User(userId)` ile kişiye gönderilir. Kullanıcının birden fazla sekmesi veya cihazı varsa hepsine ulaşır. Rapora özel grup yoktur; istemci gelen mesajdaki `reportId` ile açık ekranı günceller.
- **Soyutlama:** Application katmanı SignalR'ı bilmez, `IRealtimePublisher` interface'ini kullanır. WebApi'deki `SignalRRealtimePublisher` bunu `IHubContext<NotificationHub>` ile uygular.

Olay adı	Alıcı	İçerik	İstemcideki etkisi
`notification.created`	İlgili kullanıcı	id, type, title, body, payload, createdAt	Pop-up + okunmamış sayısı artar
`offer.created`	Rapor sahibi müşteri	reportId, offer (liste öğesi)	Teklif listesine canlı eklenir
`offer.updated`	Rapor sahibi müşteri	reportId, offerId, totalPrice, estimatedDays	Liste öğesi güncellenir
`offer.withdrawn`	Rapor sahibi müşteri	reportId, offerId	Öğe listeden düşer
`report.statusChanged`	Müşteri + teklif vermiş ustalar	reportId, status, selectedOfferId	Açık ekran yeniden yüklenir, butonlar durumuna göre açılır/kapanır

Bağlantı kopması: İstemci `withAutomaticReconnect()` ile yeniden bağlanır; yeniden bağlandığında TanStack Query açık sorguları yeniler. Bu sayede kopukluk sırasında kaçan olaylar veri kaybına yol açmaz, çünkü asıl kaynak her zaman REST API'dir.

13. Infrastructure katmanı

Infrastructure, Application'daki 11 altyapı ve 11 repository interface'inin tek gerçek implementasyonunu içerir. Burada iş kuralı yoktur; yalnızca teknolojiye bağlı kod vardır.

13.1 Kalıcılık

- `AppDbContext : IdentityDbContext<ApplicationUser, ApplicationRole, Guid>` hem Identity tablolarını hem uygulama tablolarını yönetir.

- Her entity için ayrı bir `IEntityTypeConfiguration<T>` sınıfı `Persistence/Configurations` altında durur. Kolon uzunlukları, `CHECK` kısıtları (`HasCheckConstraint`), kısmi indeksler (`HasFilter`) ve `xmin` eşzamanlılık belirteci burada tanımlanır.
- Enum'lar `HasConversion<string>()` ile metin olarak saklanır.
- Repository'ler `Persistence/Repositories` altında, her biri kendi interface'ini uygular. Liste sorguları `AsNoTracking()` ve doğrudan response modeline projeksiyon (`Select`) kullanır; gereksiz kolon çekilmez.
- `UnitOfWork.SaveChangesAsync`: olayları topla → kaydet → `DbUpdateException` içindeki PostgreSQL `23505` kodunu `ConflictException`'a çevir → olayları kuyruğa ver.
- Eşleştirme sorgusu (şehir stratejisi) tek SQL'e iner: `mechanic_profiles` + `mechanic_specialties` join, `city = @city AND approval_status = 'Approved' AND is_available AND damage_category_id = ANY(@rootIds)`. Mesafe stratejisi aynı sorguya Haversine hesabı ekler; PostGIS gerekmez.

13.2 Adapter'lar ve yan servisler

Interface	Implementasyon	Not
`IIdentityService`	`IdentityService`	`UserManager`, `SignInManager`, `RoleManager` sarmalayıcı
`ITokenService`	`JwtTokenService`	HMAC-SHA256; claim'ler: sub, email, name, role, mechanic_status
`ICurrentUser`	`CurrentUser`	`IHttpContextAccessor` üzerinden; arka plan işlerinde `SystemUser`
`IDateTimeProvider`	`SystemDateTimeProvider`	Singleton; testte sabit saatli sahte sürüm
`IFileStorage`	`LocalFileStorage`	Kök klasör `Storage:RootPath`; anahtar içinde `..` reddedilir (path traversal)
`IImageProcessor`	`ImageSharpProcessor` → `ResizeProcessor` → `CompressionProcessor` → `ExifStripProcessor`	Decorator zinciri DI'da sarılarak kurulur; sıra ayardan değil koddan

Interface	Implementasyon	Not
`INotificationChannel` (Email)	`SmtpEmailChannel` + `ResilientNotificationChannel`	MailKit; şablonlar `Notifications/Templates` altında
`INotificationChannel` (Sms)	`LoggingSmsChannel` + `ResilientNotificationChannel`	Sağlayıcı seçilene kadar yalnızca loglar ve `Skipped` yazar
`IEventDispatcher`	`InProcessEventDispatcher`	`Channel<IDomainEvent>` (unbounded)
`IRealtimePublisher`	`SignalRRealtimePublisher` (WebApi'de)	Hub WebApi'de olduğu için implementasyon da orada

13.3 Dayanıklılık ayarları (Polly)

- **Retry:** 3 deneme, üstel bekleme (1 s, 2 s, 4 s) + rastgele sapma. Yalnızca geçici hatalarda (zaman aşımı, SMTP 4xx) tekrar dener.
- **Circuit Breaker:** 30 saniyelik pencerede en az 5 çağrının %50'si başarısızsa devre 60 saniye açılır. Açıkken çağrı hiç yapılmaz, sonuç `CircuitOpen` olarak kaydedilir.
- **Timeout:** Tek gönderim 10 saniye.
- Değerlerin tamamı `Resilience:Email` bölümünden okunur.

13.4 Arka plan işleri

İş	Sıklık	Görev
`EventProcessingBackgroundService`	Sürekli	Olay kuyruğunu işler (Bölüm 9.1)
`ReportExpirationJob`	5 dakikada bir (`PeriodicTimer`)	`expires_at < now` olan `Open` raporları 100'lük gruplar halinde `Expire()` eder
`TemporaryUploadCleanupJob`	Saatte bir	24 saatten eski `temp/` dosyalarını siler
`RefreshTokenCleanupJob`	Günde bir	Süresi dolmuş veya iptal edilmiş token'ları siler

Tek sunucuda çalışıldığı için işler aynı API sürecinde `BackgroundService` olarak koşar. Yatay ölçeklemede aynı işin iki kez çalışmaması için PostgreSQL advisory lock eklenir (Future Work).

13.5 `appsettings.json` iskeleti

```
{
  "ConnectionStrings": { "Default":
"Host=db;Database=industryapp;Username=industryapp;Password=__ENV__" },
  "Jwt": { "Issuer": "industryapp", "Audience": "industryapp-clients",
"SigningKey": "__ENV__",
           "AccessTokenMinutes": 30, "RefreshTokenDays": 14 },
  "Cors": { "AllowedOrigins": [ "https://industryapp.com" ] },
  "Storage": { "RootPath": "/data/uploads" },
  "Photos": { "MaxCountPerReport": 8, "MaxFileSizeMb": 10, "MaxWidthPx":
1920, "JpegQuality": 80,
              "AllowedContentTypes": [ "image/jpeg", "image/png",
"image/heic", "image/webp" ] },
  "Reports": { "ExpireAfterHours": 72, "MaxOpenPerCustomer": 3 },
  "Offers": { "SelectionUndoWindowMinutes": 10 },
  "Matching": { "Strategy": "CITY", "DefaultRadiusKm": 15,
"MaxMechanicsPerReport": 20,
                "Filters": [ "ApprovedAndActive", "Specialty",
"Availability", "DailyCapacity" ] },
  "Email": { "Host": "__ENV__", "Port": 587, "UseStartTls": true, "User":
"__ENV__", "Password": "__ENV__",
             "FromAddress": "bildirim@industryapp.com", "FromName":
"IndustryApp" },
  "Resilience": { "Email": { "RetryCount": 3, "BreakDurationSeconds": 60,
"FailureRatio": 0.5,
                           "SamplingSeconds": 30, "MinimumThroughput": 5,
"TimeoutSeconds": 10 } },
  "RateLimiting": { "LoginPerMinute": 5, "RegisterPerMinute": 3,
"UploadPerMinute": 30 },
  "Serilog": { "MinimumLevel": { "Default": "Information", "Override": {
"Microsoft": "Warning" } } }
}
```

`__ENV__` değerleri dosyada tutulmaz; üretimde ortam değişkeniyle (`Jwt__SigningKey` gibi) verilir. `Matching:Filters` listesi CoR zincirinin sırasını belirler; filtre sırasını değiştirmek kod değişikliği gerektirmez.

13.6 DI kayıtları (özet)

```
// Application/DependencyInjection.cs
services.AddScoped<IAuthService, AuthService>();
services.AddScoped<IReportService, ReportService>();
services.AddScoped<IOfferSelectionService, OfferSelectionService>();
// ... diğer servisler aynı kalıpla
services.AddScoped<IUserProfileFactory, CustomerProfileFactory>();
services.AddScoped<IUserProfileFactory, MechanicProfileFactory>();      //
IEnumerable ile role göre seçilir
services.AddScoped<IMatchingStrategy, CityMatchingStrategy>();
services.AddScoped<IMatchingStrategy, RadiusMatchingStrategy>();        //
Matching:Strategy ile seçilir
services.AddScoped<IMechanicFilter, ApprovedAndActiveFilter>();
services.AddScoped<IMechanicFilter, SpecialtyFilter>();                 //
Matching:Filters sırasıyla zincirlenir
services.Scan(s => s.FromAssemblyOf<IAuthService>()                     //
Scrutor: tüm IEventHandler<T>
    .AddClasses(c =>
c.AssignableTo(typeof(IEventHandler<>))).AsImplementedInterfaces().WithScoped
Lifetime());
services.AddValidatorsFromAssemblyContaining<IAuthService>();

// Infrastructure/DependencyInjection.cs
services.AddSingleton<IDateTimeProvider, SystemDateTimeProvider>();
services.AddSingleton<InProcessEventDispatcher>();
services.AddSingleton<IEventDispatcher>(sp =>
sp.GetRequiredService<InProcessEventDispatcher>());
services.AddScoped<IFileStorage, LocalFileStorage>();
services.AddScoped<IImageProcessor>(sp =>                                //
Decorator zinciri
    new ExifStripProcessor(new CompressionProcessor(new ResizeProcessor(new
ImageSharpProcessor(), ...), ...)));
services.AddHostedService<EventProcessingBackgroundService>();
services.AddHostedService<ReportExpirationJob>();
```

Not: `IEnumerable<IEventHandler<T>>` çözümlemesi için Scrutor paketi eklenir; alternatifi her handler'ı elle kaydetmektir, ancak yeni handler eklendiğinde kayıt unutulursa olay sessizce işlenmez.

14. Frontend: React + TypeScript

Frontend 11 modül, 20 view ve 4 layout'tan oluşur. MVC katmanları her modülün içinde klasörle ayrılır ve tek yönlü bağımlılık kuralı backend'deki gibi uygulanır: View → Controller → Model → API.

14.1 MVC eşlemesi

Katman	Klasör	İçerik	Kural
Model	`modules/<modül>/model/`	TypeScript tipleri (response/request), `<modül>Api.ts` (Axios çağrıları), Zod şemaları, enum etiket haritaları	UI bilmez; React import etmez
Controller	`modules/<modül>/controller/`	Custom hook'lar: `useCreateReport`, `useReportOffers`, `useSelectOffer`; TanStack Query sorgu/mutasyonları, SignalR olay aboneliği, form durumu	View'a yalnızca veri ve eylem fonksiyonu verir
View	`modules/<modül>/views/` ve `components/`	Sayfalar ve MUI bileşenleri	Axios, `fetch` veya SignalR'ı doğrudan çağırmaz; yalnızca hook kullanır

Bu ayrım sayesinde ileride mobil uygulama yazılırken `model/` klasörleri (tipler ve API çağrıları) doğrudan paylaşılabilir; yalnızca view'lar yeniden yazılır.

14.2 Klasör yapısı

```
industryapp-web/
├── src/
|   ├── app/
|   |   ├── App.tsx                 → provider'lar (QueryClient, Theme,
Auth, Realtime)
|   |   ├── router.tsx              → tüm route'lar ve guard'lar
|   |   └── layouts/                → PublicLayout, CustomerLayout,
MechanicLayout, AdminLayout
|   ├── shared/
|   |   ├── http/                   → axiosClient (token ekleme, 401'de
refresh, ProblemDetails çözümleme)
|   |   ├── realtime/               → RealtimeProvider,
useRealtimeEvent(eventName, handler)
|   |   ├── auth/                   → AuthProvider, useAuth, RequireRole
```

```
guard, tokenStorage
|   |   ├── ui/                       → ortak bileşenler: PageHeader,
EmptyState, ConfirmDialog, StatusChip
|   |   ├── format/                   → formatDate, formatMoney, formatPhone,
labelOf
|   |   └── errors/                   → hata kodu → Türkçe mesaj haritası
|   └── modules/
|       ├── auth/                     → giriş, kayıt
|       ├── account/                  → dashboard, profil, müsaitlik
|       ├── catalog/                  → hasar ağacı, araç türleri, sözleşmeler
(paylaşılan hook'lar)
|       ├── reports/                  → müşteri raporları, sihirbaz, fotoğraf
yükleme
|       ├── mechanicJobs/             → ustaya gelen işler
|       ├── offers/                   → teklif formu, teklif listesi, sıralama
|       ├── selection/                → teklif seçimi, geri alma sayacı
|       ├── contacts/                 → eşleşme ekranı
|       ├── notifications/            → bildirim paneli ve sayfası
|       ├── admin/                    → usta onayı, kategori yönetimi
|       └── system/                   → 403/404 sayfaları
├── .env.example                      → VITE_API_BASE_URL, VITE_HUB_URL
├── vite.config.ts
└── Dockerfile                        → build + Caddy ile servis
```

14.3 View listesi ve route'lar

#	View	Route	Yetki	Modül	UC
1	`LoginView`	`/login`	Anonim	auth	UC-01
2	`RegisterRoleSelectView`	`/register`	Anonim	auth	UC-01
3	`RegisterCustomerView`	`/register/customer`	Anonim	auth	UC-01
4	`RegisterMechanicView`	`/register/mechanic`	Anonim	auth	UC-01
5	`ProfileView`	`/profile`	Giriş yapmış	account	UC-01
6	`NotificationsView`	`/notifications`	Giriş yapmış	notifications	Tümü
7	`MatchView`	`/match/:reportId`	Müşteri veya eşleşen usta	contacts	UC-04
8	`ErrorView`	`/403`, `*`	Herkes	system	—
9	`CustomerDashboardView`	`/customer`	Customer	account	UC-01
10	`MyReportsView`	`/customer/reports`	Customer	reports	UC-02
11	`CreateReportWizardView`	`/customer/reports/new`	Customer	reports	UC-02
12	`CustomerReportDetailView`	`/customer/reports/:reportId`	Customer	reports + offers + selection	UC-04
13	`MechanicDashboardView`	`/mechanic`	Mechanic (onaylı)	account	UC-01

#	View	Route	Yetki	Modül	UC
14	`MechanicPendingApprovalView`	`/mechanic/pending`	Mechanic (onaysız)	account	UC-01
15	`JobsView`	`/mechanic/jobs`	Mechanic (onaylı)	mechanicJobs	UC-03
16	`JobDetailView`	`/mechanic/jobs/:reportId`	Mechanic (onaylı)	mechanicJobs + offers	UC-03
17	`MyOffersView`	`/mechanic/offers`	Mechanic (onaylı)	offers	UC-03
18	`AdminDashboardView`	`/admin`	Admin	admin	YA-01
19	`MechanicApprovalsView`	`/admin/mechanics`	Admin	admin	YA-01
20	`CategoryManagementView`	`/admin/categories`	Admin	admin	YA-01

Giriş sonrası yönlendirme `useAuth().role` değerine göre yapılır: Customer → `/customer`, onaylı usta → `/mechanic`, onaysız usta → `/mechanic/pending`, Admin → `/admin`. Layout'un menüsü `/api/account/dashboard` yanıtından gelir; bu, backend'deki Abstract Factory'nin frontend'deki karşılığıdır.

14.4 Öne çıkan bileşenler

- **`CreateReportWizardView`**: MUI `Stepper` ile 5 adım: `PhotoUploadStep`, `DamageSelectStep` (hasar ağacı, `TreeView`), `VehicleStep`, `LocationStep` (tarayıcı konumu veya il/ilçe), `SummaryStep`. Form durumu React Hook Form ile adımlar arasında korunur; fotoğraflar seçildiği anda `POST /api/photos/temporary` ile yüklenir ve ilerleme çubuğu gösterilir.
- **`CustomerReportDetailView`**: Üstte rapor özeti ve durum çipi; altında `OfferList` (sıralama seçimi, fiyat kırılımı açılır paneli, "Bu teklifi seç" butonu). Rapor `OfferSelected` olunca `SelectionBanner` geri alma geri sayımını ve "Eşleşmeye git" butonunu gösterir. `offer.created` ve `report.statusChanged` olaylarıyla canlı güncellenir.
- **`JobDetailView`**: Rapor fotoğrafları ve hasar listesi; `OfferForm` (fiyat kalemleri dinamik liste, toplam anlık hesaplanır, tahmini gün, not). Usta zaten teklif verdiyse form güncelleme moduna geçer ve "Teklifi geri çek" butonu çıkar.
- **`NotificationBell`**: Layout'ta; okunmamış sayısı `notification.created` ile artar, tıklanınca son 10 bildirim açılır; bildirime tıklamak `payload` içindeki `reportId` ile ilgili view'a gider.

14.5 Veri biçimlendirme kuralları

- Tarih `dd/MM/yyyy`, tarih-saat `dd/MM/yyyy HH:mm` biçiminde gösterilir; değer yoksa `'-'` yazılır. Tek kaynak `shared/format/formatDate.ts`.

- Enum ve kod değerleri (`status` , `type` , `severity`) `labelOf(map, value)` ile Türkçe etikete çevrilir; eşleşme yoksa veya değer boşsa `'-'` gösterilir, ham kod ekrana çıkmaz.
- Para `formatMoney(amount, currency)` ile `12.500,00 ₺` biçiminde gösterilir.
- Backend REST ve SignalR yanıtları camelCase'e sabitlenir (`AddJsonProtocol` dahil). Böylece frontend'de aynı alanı hem camelCase hem PascalCase aramaya gerek kalmaz; tipler tek biçimle yazılır.

14.6 Token ve oturum

Access token bellekte, refresh token `localStorage` 'da tutulur. Axios interceptor'ı 401 aldığında tek bir refresh isteği atar, bekleyen istekleri kuyruğa alır ve yeni token ile tekrarlar. Refresh başarısızsa oturum kapatılıp `/login` 'e yönlendirilir. **Risk notu:** Refresh token'ın `localStorage` 'da tutulması XSS'e karşı `httpOnly` cookie kadar güvenli değildir; ilk sürüm için bilinçli bir sadeleştirmedir ve Future Work'te cookie tabanlı yaklaşıma geçilmesi önerilir.

15. Güvenlik ve üretim hazırlığı

Proje ilk günden gerçek kullanıcı verisiyle çalışacağı için aşağıdaki maddelerin tamamı yayından önce tamamlanır.

Alan	Önlem
Aktarım	Yalnızca HTTPS (Caddy otomatik Let's Encrypt sertifikası); HSTS başlığı
Şifre	Identity politikası: en az 8 karakter, rakam ve harf; 5 hatalı denemede 15 dk kilit
Token	Access 30 dk; refresh 14 gün, her kullanımda yenilenir (rotation); iptal edilmiş token tekrar kullanılırsa kullanıcının tüm token'ları iptal edilir (çalıntı token tespiti); veritabanında yalnızca SHA-256 özeti tutulur
Gizli anahtarlar	JWT anahtarı, veritabanı ve SMTP şifreleri yalnızca ortam değişkeninde; repoda `.env` yoktur, yalnızca `.env.example`
CORS	Yalnızca `Cors:AllowedOrigins` listesindeki alan adları

Alan	Önlem
İstek sınırlama	.NET 8 `RateLimiter`: giriş 5/dk/IP, kayıt 3/dk/IP, fotoğraf 30/dk/kullanıcı
Dosya yükleme	İçerik türü başlığına güvenilmez, dosyanın ilk baytları (magic number) kontrol edilir; boyut sınırı; dosya adı kullanıcıdan alınmaz, sunucu GUID üretir; EXIF temizlenir
Dosya erişimi	Fotoğraflar statik klasörden değil `GET /api/photos/{id}` ile, yetki kontrolünden geçerek sunulur
Yetki	Rol kontrolü attribute ile, kaynak sahipliği serviste; kapalı teklif kuralı response modelinin şekliyle
İletişim bilgisi	Telefon ve e-posta yalnızca `ContactInfoProxy` üzerinden; loglarda telefon maskelenir (`05** *** ** 12`)
Girdi doğrulama	FluentValidation her request için; EF Core parametreli sorgu (SQL injection yok); React çıktıyı kaçışlar (XSS)
Loglama	Serilog; her istek `traceId` ile; şifre, token ve tam telefon numarası loglanmaz
KVKK	Kayıtta aydınlatma metni onayı ve sürümü saklanır; hesap silme talebinde kişisel alanlar anonimleştirilir (Future Work'te self-servis)
Sağlık kontrolü	`GET /health` (veritabanı bağlantısı dahil); Docker healthcheck bunu kullanır
Yedekleme	Günlük `pg_dump` + yükleme klasörünün yedeği, 7 gün saklama
Migration	Üretimde uygulama açılışında otomatik migration yapılmaz; dağıtım adımında `migrations bundle` çalıştırılır. Böylece hatalı migration uygulamayı ayağa kalkarken kilitlemez

Yayın öncesi kontrol listesi:

- [ ] Alan adı alındı, DNS sunucuya yönlendirildi
- [ ] `.env` üretim değerleriyle dolduruldu, JWT anahtarı en az 64 rastgele karakter
- [ ] SMTP hesabı (ör. kurumsal e-posta veya işlem e-postası sağlayıcısı) test edildi

- [ ] Migration bundle ve `reference-data.sql` çalıştırıldı
- [ ] İlk admin oluşturuldu ve `admin-bootstrap.sql` çalıştırıldı
- [ ] Sözleşme metinlerinin gerçek içerikleri `legal_documents` tablosuna girildi
- [ ] Yedekleme görevi kuruldu ve bir geri yükleme denendi

16. Dağıtım

Üretim tek bir Linux sunucuda (2 vCPU, 4 GB RAM yeterli) Docker Compose ile 3 konteyner olarak çalışır: `db`, `api`, `web`. `web` konteyneri Caddy'dir; hem derlenmiş React dosyalarını sunar hem de `/api` ve `/hubs` isteklerini API'ye iletir. Tek alan adı kullanıldığı için CORS ve cookie sorunları da ortadan kalkar.

16.1 `docker-compose.yml`

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: industryapp
      POSTGRES_USER: industryapp
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U industryapp"]
      interval: 10s

  api:
    build: ./src/backend
    restart: unless-stopped
    depends_on:
      db: { condition: service_healthy }
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ConnectionStrings__Default:
Host=db;Database=industryapp;Username=industryapp;Password=${DB_PASSWORD}
      Jwt__SigningKey: ${JWT_SIGNING_KEY}
      Email__Host: ${SMTP_HOST}
      Email__User: ${SMTP_USER}
      Email__Password: ${SMTP_PASSWORD}
      Cors__AllowedOrigins__0: https://${DOMAIN}
    volumes:
      - uploads:/data/uploads
```

```
      - logs:/app/logs

  web:
    build:
      context: ./src/frontend/industryapp-web
      args:
        VITE_API_BASE_URL: https://${DOMAIN}/api
        VITE_HUB_URL: https://${DOMAIN}/hubs/notifications
    restart: unless-stopped
    depends_on: [api]
    ports: ["80:80", "443:443"]
    environment:
      DOMAIN: ${DOMAIN}
    volumes:
      - caddy_data:/data

volumes:
  pgdata:
  uploads:
  logs:
  caddy_data:
```

16.2 `deploy/Caddyfile`

```
{$DOMAIN} {
    encode gzip
    handle /api/*  { reverse_proxy api:8080 }
    handle /hubs/* { reverse_proxy api:8080 }
    handle /health { reverse_proxy api:8080 }
    handle {
        root * /srv
        try_files {path} /index.html
        file_server
    }
    header Strict-Transport-Security "max-age=31536000"
}
```

16.3 Ortamlar

Ortam	Nasıl çalışır	Veritabanı
Geliştirme	API `dotnet watch`, frontend `npm run dev` (Vite proxy ile `/api` → localhost:5080), yalnızca `db` konteyneri Docker'da	Yerel PostgreSQL konteyneri; geliştiriciler kendi test kayıtlarını ekranlardan oluşturur
Üretim	`docker compose up -d --build`	Sunucudaki `pgdata` volume'ü

16.4 Dağıtım adımları

1. Sunucuya Docker kurulur, repo klonlanır, `deploy/.env.example` → `.env` olarak kopyalanıp doldurulur.
2. `docker compose up -d db` ile veritabanı başlatılır.
3. Migration bundle çalıştırılır: `docker compose run --rm api ./efbundle --connection "$CONN"`.
4. `psql` ile `database/reference-data.sql` çalıştırılır.
5. `docker compose up -d --build` ile tüm servisler başlatılır.
6. Uygulamadan admin adayı kaydolur, `database/admin-bootstrap.sql` çalıştırılır.
7. Güncellemelerde: `git pull` → yeni migration varsa adım 3 → `docker compose up -d --build api web`.

17. Adım adım uygulama planı

Proje 9 fazda kurulur. Her faz bir önceki bitmeden başlamaz ve sonunda çalışan, commit edilmiş bir durum bırakır. UC-04 PA2 için erken yapılır; o aşamada UC-02 ve UC-03 henüz olmadığından yalnızca yerel geliştirme veritabanında `database/dev/uc04-dev-data.sql` ile rapor ve teklif eklenir. Bu betik üretimde asla çalıştırılmaz ve F6 bitince silinir.

F0: Hazırlık

- [ ] Repo oluştur; `.gitignore` (.NET + Node), `.editorconfig`, `README.md` iskeleti
- [ ] `src/backend` altında solution ve 4 proje: `dotnet new classlib` (Domain, Application, Infrastructure), `dotnet new webapi` (WebApi), 2 xUnit test projesi
- [ ] Proje referanslarını Bölüm 3.1'e göre ekle; `Directory.Build.props` ile `net8.0`, `Nullable=enable`, `TreatWarningsAsErrors=true`
- [ ] Paketleri ekle (Bölüm 2): Npgsql EF, NamingConventions, Identity, JwtBearer, FluentValidation, Polly, ImageSharp, MailKit, Serilog, Swashbuckle, Scrutor

- [ ] `docker-compose.yml`'de yalnızca `db` servisiyle yerel PostgreSQL'i başlat
- [ ] `src/frontend` altında `npm create vite@latest industryapp-web -- --template react-ts`; MUI, React Router, TanStack Query, Axios, React Hook Form, Zod, `@microsoft/signalr` kur
- [ ] Vite proxy: `/api` ve `/hubs` → `http://localhost:5080`

F1: Domain çekirdeği

- [ ] `Common`: `BaseEntity`, `AuditableEntity`, `IDomainEvent`, `IHasDomainEvents`, `DomainException`
- [ ] Accounts: `CustomerProfile`, `MechanicProfile` (+ `Approve`, `Reject`, `SetAvailability`), `MechanicSpecialty`, `LegalDocument`, `UserConsent`, `RoleNames`
- [ ] Catalog: `IDamageComponent`, `DamageCategory` (Composite), `VehicleType`
- [ ] Reports: `AccidentReport`, `AccidentReportBuilder`, `ReportDamageItem`, `ReportPhoto`, `ReportMatch`, `ReportStatusHistory`, `GeoLocation`
- [ ] States: `IReportState` + 5 durum sınıfı + `ReportStateFactory`
- [ ] Offers: `Offer`, `OfferPriceItem`, Pricing Decorator zinciri
- [ ] Contacts: `IContactInfo`, `ContactInfo`, `ContactInfoProxy`
- [ ] Notifications: `Notification`; Events: 12 olay record'u
- [ ] Birim testleri: State geçişleri, Builder eksik alan, Proxy maskeleme, fiyat toplamı

F2: Kalıcılık ve olay altyapısı

- [ ] Infrastructure/Identity: `ApplicationUser`, `ApplicationRole`
- [ ] `AppDbContext`, Identity tablo adlarını sadeleştir, `UseSnakeCaseNamingConvention()`
- [ ] Her entity için `IEntityTypeConfiguration`; kısıtlar ve indeksler Bölüm 6'ya göre
- [ ] `dotnet ef migrations add Initial`; `migrations script` çıktısını Bölüm 6 DDL'iyle karşılaştır
- [ ] `database/reference-data.sql` yaz ve yerel veritabanında çalıştır
- [ ] Application/Abstractions interface'leri ve 11 repository interface'i
- [ ] Repository implementasyonları, `UnitOfWork`
- [ ] `InProcessEventDispatcher`, `EventProcessingBackgroundService`
- [ ] `Result`, `Error`, `ApiControllerBase`, `GlobalExceptionHandler`, Serilog, Swagger

F3: Kimlik ve iskelet (UC-01 temel)

- [ ] `IdentityService`, `JwtTokenService`, refresh token tablosu ve rotasyonu, `CurrentUser`

- [ ] Yetki politikaları (Bölüm 11.2)
- [ ] `IAuthService` ve `AuthController`: kayıt (müşteri, usta), giriş, refresh, çıkış
- [ ] Frontend `shared/`: axiosClient (token + refresh), AuthProvider, RequireRole, layout'lar, format yardımcıları, hata kodu haritası
- [ ] View'lar: `LoginView`, `RegisterRoleSelectView`, `RegisterCustomerView`, `RegisterMechanicView`, `ErrorView`
- [ ] Doğrulama: ekrandan bir müşteri ve bir usta kaydol; ustayı yerelde SQL ile `Approved` yap

F4: UC-04 Teklif seçimi (PA2)

- [ ] `database/dev/uc04-dev-data.sql`: F3'te kaydolan kullanıcılar için 1 rapor ve 3 teklif (yalnızca yerel)
- [ ] `IOfferSortStrategy` + 3 sıralama; `IOfferService.ListForReportAsync`
- [ ] `SelectOfferCommand`, `IReportCommandFactory`, `IOfferSelectionService` (seç, geri al, durum)
- [ ] `IContactService` + `ContactInfoProxy` kullanımı
- [ ] Controller'lar: `OffersController` (liste), `OfferSelectionController`, `ContactsController`, `ReportsController` (detay, tamamla, iptal)
- [ ] SignalR hub, `SignalRRealtimePublisher`, `NotificationService` (uygulama içi), `OfferSelectedNotificationHandler`
- [ ] Frontend: `CustomerReportDetailView` (teklif listesi, sıralama, seçim, geri alma sayacı), `MatchView`, `NotificationBell`
- [ ] Desen yorumları (Bölüm 10.1), birim ve entegrasyon testleri
- [ ] PA2 Software Design Document ve README

F5: UC-02 Kaza raporu

- [ ] `ICatalogService` + `CatalogController`
- [ ] Görsel işleme Decorator zinciri, `LocalFileStorage`, `IPhotoService`, `PhotosController`
- [ ] `IReportService.CreateAsync` (Builder, geçici fotoğraf taşıma), `ListMineAsync`
- [ ] `TemporaryUploadCleanupJob`, `ReportExpirationJob`
- [ ] Frontend: `CreateReportWizardView` (5 adım), `MyReportsView`, `CustomerDashboardView`

F6: UC-03 Eşleştirme ve teklif verme + usta onayı

- [ ] `IMatchingStrategy` (City, Radius), `IMechanicFilter` zinciri (4 filtre), `IMatchingService`
- [ ] `AccidentReportCreatedMatchingHandler`, `MechanicsMatchedNotificationHandler`
- [ ] `SmtpEmailChannel`, `LoggingSmsChannel`, `ResilientNotificationChannel` (Polly), `notification_deliveries` kaydı
- [ ] `IOfferService` teklif ver / güncelle / geri çek; `OffersController`, `MechanicJobsController`
- [ ] `IMechanicApprovalService`, `AdminMechanicsController`
- [ ] Frontend: `JobsView`, `JobDetailView` (OfferForm), `MyOffersView`, `MechanicDashboardView`, `MechanicPendingApprovalView`, `MechanicApprovalsView`
- [ ] Uçtan uca: müşteri rapor açar → usta pop-up alır → teklif verir → müşteri seçer; `uc04-dev-data.sql` artık gereksiz, sil

F7: UC-01 tamamlama ve yönetim

- [ ] `IUserProfileFactory` (Factory Method) ile kaydın yeniden düzenlenmesi
- [ ] `IDashboardFactory` ailesi (Abstract Factory), `GET /api/account/dashboard`
- [ ] Sözleşme onayı akışı, `user_consents`
- [ ] Profil düzenleme, müsaitlik anahtarı: `ProfileView`
- [ ] `ICategoryAdminService`, `AdminCatalogController`, `CategoryManagementView`, `AdminDashboardView`
- [ ] `NotificationsView`

F8: Sağlamlaştırma ve yayın

- [ ] Rate limiting, CORS, HSTS, dosya magic number kontrolü, `/health`
- [ ] Tüm testler yeşil; test raporu Excel'i (Bölüm 19)
- [ ] Backend ve frontend Dockerfile'ları, Caddyfile, `.env.example`
- [ ] Sunucuya ilk dağıtım (Bölüm 16.4) ve yayın öncesi kontrol listesi (Bölüm 15)
- [ ] PA3 dokümanları, demo senaryosunun üretim ortamında provası

18. İzlenebilirlik ve PA eşlemesi

Her iş parçası bir UC numarasına bağlanır ve bu numara dokümandan koda, commit'ten teste kadar değişmeden taşınır. Hocanın istediği traceability bu zincirle sağlanır.

18.1 Numaralandırma zinciri

Öğe	Biçim	Örnek
Use case	`UC-0n`	UC-04
Fonksiyonel gereksinim	`FR-0n.m`	FR-04.3 "Müşteri seçimi süre içinde geri alabilir"
Desen	`PAT-0n.m`	PAT-04.2 Command
Kod yorumu	`// [UC-0n][PAT-0n.m] <Desen>`	`// [UC-04][PAT-04.2] Command`
Test	`TC-0n.m.k` + test metot adında	`TC-04.3.1 Undo_WithinWindow_RestoresPendingOffers`
Commit	`[UC-0n] <tür>: <açıklama>`	`[UC-04] feat: teklif seçimi geri alma`
Branch	`uc-0n/<kısa-ad>`	`uc-04/undo-selection`

18.2 PA teslimlerine eşleme

Teslim	Bu dokümanın kaynak bölümleri	Ek iş
PA1 Project Definition	1, 0.2	Başarı ölçütleri ve teslimatlar
PA1 Project Plan	17, 15, 16, 2	Gantt (fazlar), risk tablosu, iletişim planı, değişiklik yönetimi, bütçe/efor
PA1 SRS	7, 1, 15, 11.3	UML use case diyagramı, fonksiyonel olmayan gereksinimler
PA1 Mimari + Desen matrisi	4, 10	Mimari diyagram
PA2 SDD (UC-04)	4, 5, 6, 8, 9, 10, 11, 12, 13, 14, 16	UML sınıf, sekans ve bileşen diyagramları, ER diyagramı, UI ekran görüntüleri
PA2 kaynak kod	F0–F4	`TASK_MATRIX.md`, README
PA3 güncel SDD	Tüm bölümler	UC-01..03 diyagramları
PA3 test raporu	19	Excel

Teslim	Bu dokümanın kaynak bölümleri	Ek iş
PA3 kaynak kod	F5–F8	Demo üretim ortamında

18.3 Fonksiyonel gereksinimler

Kod	Gereksinim
FR-01.1	Kullanıcı müşteri veya usta olarak role özel formla kaydolur
FR-01.2	Kayıtta role özel sözleşmelerin onayı zorunludur ve sürümüyle saklanır
FR-01.3	Usta hesabı admin onayına kadar iş alamaz
FR-01.4	Giriş sonrası kullanıcı rolüne özel panele yönlenir
FR-02.1	Müşteri 1–8 fotoğraf yükler; fotoğraflar boyutlandırılır ve konum verisi silinir
FR-02.2	Müşteri hasar ağacından bir veya daha fazla bölge ve şiddet seçer
FR-02.3	Eksik bilgiyle rapor oluşturulamaz
FR-02.4	Rapor, teklif seçilmezse ayarlı süre sonunda kendiliğinden kapanır
FR-03.1	Rapor oluşunca şehir ve uzmanlığa uyan, onaylı ve müsait ustalar otomatik bulunur
FR-03.2	Eşleşen ustalar uygulama içi ve e-posta ile bilgilendirilir
FR-03.3	Usta fiyat kırılımıyla tek teklif verir, güncelleyebilir veya geri çekebilir
FR-03.4	Usta diğer ustaların tekliflerini göremez
FR-03.5	E-posta sağlayıcısı çökerse uygulama içi bildirimler etkilenmez
FR-04.1	Müşteri teklifleri anlık görür ve fiyat, süre veya yeniliğe göre sıralar
FR-04.2	Müşteri tek bir teklif seçer; diğer teklifler seçilmedi olarak işaretlenir

Kod	Gereksinim
FR-04.3	Müşteri seçimi ayarlı süre içinde geri alabilir
FR-04.4	İletişim bilgileri yalnızca müşteri ile seçilen ustaya açılır
FR-04.5	Müşteri işi tamamlandı olarak işaretler veya raporu iptal eder

19. Test planı

Testler üç seviyededir: domain ve application birim testleri (veritabanısız), gerçek PostgreSQL konteyneriyle entegrasyon testleri ve üretim benzeri ortamda elle yürütülen uçtan uca senaryolar. PA3'teki Excel test raporu üçüncü seviyeden beslenir.

19.1 Otomatik testler

Kod	Seviye	Test	Doğruladığı
TC-02.3.1	Birim	`Builder_MissingPhoto_Throws`	PAT-02.1, FR-02.3
TC-02.1.1	Birim	`ImagePipeline_RemovesGpsExif`	PAT-02.3, FR-02.1
TC-03.1.1	Birim	`FilterChain_ExcludesUnapprovedAndBusy`	PAT-03.3, FR-03.1
TC-03.1.2	Birim	`StrategyResolver_UsesConfiguredStrategy`	PAT-03.2
TC-03.3.1	Birim	`PriceDecorators_TotalEqualsSumOfLines`	PAT-03.4
TC-03.5.1	Birim	`ResilientChannel_OpensCircuitAfterFailures`	PAT-03.6, FR-03.5
TC-04.2.1	Birim	`OpenState_SelectOffer_MovesToOfferSelected`	PAT-04.1
TC-04.2.2	Birim	`CompletedState_SelectOffer_Throws`	PAT-04.1
TC-04.3.1	Birim	`Undo_WithinWindow_RestoresPendingOffers`	PAT-04.2, FR-04.3

Kod	Seviye	Test	Doğruladığı
TC-04.3.2	Birim	`Undo_AfterWindow_Throws`	FR-04.3
TC-04.4.1	Birim	`ContactProxy_NotMatched_ReturnsMaskedPhone`	PAT-04.3, FR-04.4
TC-03.4.1	Entegrasyon	`MechanicJobDetail_NeverContainsOtherOffers`	FR-03.4
TC-04.2.3	Entegrasyon	`ConcurrentSelect_SecondRequestGets409`	Kısmi unique index
TC-02.3.2	Entegrasyon	`CreateReport_ReturnsBeforeMatchingCompletes`	PAT-03.1, performans
TC-01.3.1	Entegrasyon	`PendingMechanic_CannotSubmitOffer`	FR-01.3

19.2 Uçtan uca senaryolar (test raporuna girecek)

Kod	Senaryo	Beklenen sonuç
E2E-01	Usta kaydolur, admin onaylar, usta tekrar girer	Onay e-postası gelir; usta iş listesini görür
E2E-02	Müşteri 3 fotoğraflı rapor açar	Rapor `Open`; eşleşen ustalarda 3 sn içinde pop-up
E2E-03	İki usta teklif verir	Teklifler müşteri ekranına sayfa yenilemeden düşer; ustalar birbirini görmez
E2E-04	Müşteri teklif seçer	Kazanan "seçildiniz", diğeri "seçilmedi" bildirimi alır; eşleşme ekranında numaralar görünür
E2E-05	Müşteri 10 dk içinde vazgeçer	Rapor `Open`, teklifler `Pending`; ustalara bildirim
E2E-06	SMTP ayarı bilerek bozulur, rapor açılır	Uygulama içi bildirimler gider; `notification_deliveries` içinde `CircuitOpen` kayıtları
E2E-07	Eşleşmemiş usta `/api/reports/{id}/contact` çağırır	403 veya maskeli numara

Kod	Senaryo	Beklenen sonuç
E2E-08	Rapor 72 saat teklifsiz bekler (test için ayar 5 dk)	Rapor `Expired`, müşteriye bildirim

Excel test raporu kolonları (PA3): Test no, test adı, ilgili UC/FR, adımlar, beklenen sonuç, gerçekleşen sonuç, test eden, tarih (`dd/MM/yyyy`), ortam (tarayıcı, sürüm, sunucu), log/ `traceId`, durum (Pass / Fail / Blocked), not.

20. Future work ve açık konular

Konu	Neden şimdi değil	Yapılınca ne değişir
Transactional Outbox	İlk sürüm tek sunucu; bellek içi kuyruk yeterli	Olay kaybı sıfırlanır; `outbox_messages` tablosu ve işleyici eklenir, Application değişmez
Mobil uygulama	Önce web ile ürün doğrulanacak	Aynı API ve SignalR hub'ı kullanılır; `model/` katmanı paylaşılabilir
SMS sağlayıcısı	Maliyet ve sözleşme gerektirir	`INotificationChannel` için tek yeni sınıf
S3 / MinIO depolama	Tek sunucuda disk yeterli	`IFileStorage` için tek yeni sınıf
Puanlama ve yorum	Kapsam dışı	Sıralamaya "puana göre" stratejisi eklenir
Ödeme ve komisyon	Yasal ve finansal süreç gerektirir	Yeni modül; teklif seçimi sonrası olaya bağlanır
Uygulama içi mesajlaşma	İletişim şimdilik telefonla	SignalR hub'ına mesaj olayları eklenir
httpOnly cookie ile refresh token	İlk sürümde sadelik	XSS riskine karşı daha güvenli oturum
Çoklu sunucu	Trafik gerektirmiyor	SignalR için Redis backplane, arka plan işleri için advisory lock
KVKK self-servis hesap silme	İlk sürümde talep üzerine elle	Kullanıcı verisini anonimleştiren servis

Açık kararlar (netleşince bu doküman güncellenir):

- [ ] Alan adı ve sunucu sağlayıcısı

- [ ] E-posta gönderim hizmeti (SMTP sağlayıcısı)
- [ ] Sözleşme metinlerinin hukuki içeriği
- [ ] Varsayılan ayarların (Bölüm 0.3) son değerleri